

班 级 2003012
学 号 20009200869

西安电子科技大学

本科毕业设计论文



题 目 基于双启发式搜索算法的微服务
自动化部署策略研究

学 院 计算机科学与技术学院

专 业 计算机科学与技术

学 生 姓 名 戴 逸 飞

导 师 姓 名 杨 鹏 飞

院内导师姓名 _____

摘要

微服务架构是近年来越来越流行的一种软件开发模式，其凭借灵活性、强扩展性和高可用性等优势，为用户提供更优质的网络服务。由于微服务架构下服务分布分散、依赖多、交互复杂性高，如何在最大程度上优化部署策略成为了该领域重要研究方向。为此，本文将深入研究微服务部署策略优化问题。

本文通过分析微服务实际应用场景，构建了以微服务网络延时开销和集群负载均衡性能为目标的微服务部署策略多目标优化模型。为求解该多目标优化模型，本文研究遗传算法与粒子群算法特点，创新性地设计并实现了 MOGA-PSO 多目标双启发式算法。该算法以 NSGA 与 MOPSO 算法为参照，吸收了遗传算法快速随机的全局搜索能力与粒子群算法收敛速度快的特点，提升了算法的收敛性能与解集质量。

实验表明，MOGA-PSO 算法在收敛速度较 NSGA 优化了 78.5%，在解集质量上较 NSGA 和 MOPSO 算法也有大幅提升。在 Kubernetes 集群 Bookinfo 微服务部署实验测试中显示，MOGA-PSO 算法相较于 Kubernetes 默认策略在高并发请求下的服务响应时间显著加速，有效提升了 QoS。验证了该算法在微服务部署策略优化问题上的有效性。

关键词：微服务部署 多目标优化 双启发式算法 遗传算法 粒子群算法

ABSTRACT

Microservices architecture has become an increasingly popular software development model in recent years, providing users with better internet services through its flexibility, strong scalability, and high availability. Due to the distributed nature, multiple dependencies, and the high complexity of interactions under the microservices architecture, how to optimize the deployment strategy to the greatest extent has become an important research direction in this field. Therefore, this thesis deeply studies the optimization of the microservices deployment strategy.

This thesis constructs a multi-objective optimization model for microservices deployment strategy with microservice network latency overhead and cluster load balance performance as the objectives, by analyzing the actual application scenarios of microservices. To solve this model, this thesis studies the characteristics of genetic algorithms and particle swarm algorithms, and innovatively designs and realizes the MOGA-PSO multi-objective dual-heuristic algorithm. This algorithm takes NSGA and MOPSO algorithms as references, and absorbs the fast and random global search capabilities of genetic algorithms and the rapid convergence features of particle swarm algorithms, to improve the algorithm's convergence performance and solution set quality.

Experiments have shown that the MOGA-PSO algorithm optimizes the convergence speed by 78.5% compared to NSGA, and significantly enhances the solution set quality in comparison with both NSGA and MOPSO algorithms. In the Kubernetes cluster Bookinfo microservice deployment experiment, the MOGA-PSO algorithm, compared to the default strategy of Kubernetes in high concurrency request scenarios, observes a significant speedup in service response time, effectively improving QoS. This has verified the effectiveness of the algorithm in optimizing the deployment strategy of microservices.

Keywords: Microservice Deployment Multi-Objective Optimization Double heuristic algorithm GA PSO

目 录

第一章 绪论.....	1
1.1 研究背景及意义	1
1.2 国内外研究现状	3
1.3 研究内容	5
1.4 论文章节安排	6
第二章 相关理论及概念.....	9
2.1 多目标优化模型	9
2.2 优化算法	10
2.3 微服务相关软件	16
第三章 基于多目标优化的微服务部署模型	23
3.1 基本定义	23
3.2 目标函数	24
3.3 约束条件	26
3.4 模型构建	27
3.5 参数说明	28
第四章 基于双启发式优化的微服务部署算法设计	29
4.1 算法整体流程	29
4.2 详细设计	31
第五章 实验部分.....	37
5.1 实验设置	37
5.2 实验结果与分析	40
第六章 总结与展望.....	47
致谢.....	49
参考文献.....	51

第一章 绪论

1.1 研究背景及意义

1.1.1 研究背景

微服务架构是近年来越来越流行的一种开发模式，其受到了大量企业的关注，并被广泛采用^[1]。传统单体架构依赖于单一代码库来实现所有功能，整个应用程序在运行时由一个进程执行。单体架构软件开发简单，适用于小型应用，但是其可扩展性与容错方面弱。相比之下，微服务架构应用通过服务的精细化拆分，将传统的单体应用解耦为相互独立的小型模块化微服务，并通过标准化的通信协议实现了复杂应用的需求满足，使其在功能扩展、稳定性及应对高并发场景下更具优势。

微服务架构因其灵活性、可扩展性和高可用性，很适用于业务逻辑复杂，模块众多的大型应用服务。其灵活性体现在多个方面，如在开发者可针对不同的模块根据其特定服务需求自由选择合适的编程语言、框架及数据库；部署时，各服务能够根据各自的网络和硬件资源需求，如服务器之间的通信成本、带宽、CPU 和内存，实现灵活分布，优化资源分配。可扩展性体现在微服务可以根据用户对某一需求的实际并发请求量实时地增加或减少微服务实例，每个服务的变动独立于其他服务，避免了单体架构下不得不整体扩展业务线而造成的资源浪费。这种可扩展性不仅提高了资源效率，同时也支持了高并发的服务需求。高可用性体现在相比于单体架构软件，微服务架构的软件启动实例所需资源相对小，因此可以对关键服务进行多实例备份，以保证应用稳定性。在处理大量具有不同需求的并发请求时，因为微服务架构在灵活性、可扩展性与高可用性相对于单体架构的优势，于是以微服务架构为基础的软件逐渐开始替代单体应用。

目前，国内外许多大型互联网公司或企业（如亚马逊^[2]、Twitter^[3]、IBM^[4]、淘宝^[5]等）都开始采用基于微服务的软件架构，并利用 Docker、Kubernetes 等技术来开发和运维微服务架构软件。微服务架构已经成为在移动互联网和物联网应用中应对大量用户并发请求的有效解决方案^[6]。

虽然微服务在当今有如此蓬勃的发展，带来了分布式系统设计上空前的敏捷性和可扩展性，但是其背后的各种问题与挑战也应运而生。在微服务架构下，众多服务分散运行在不同的服务器节点上，服务之间的依赖、交互复杂性升高，如何在

最大程度上优化部署,保障服务的高可用性与性能,这便是微服务部署优化问题。为实现最优的微服务部署,需要考虑一系列因素,包括但不限于服务器硬件资源配置、网络拓扑与传输延迟,以及整体架构的可靠性和弹性。这些因素不仅自身复杂,相互间也存在交织与制约,构成了一个多变量、多约束的优化难题。在这种情况下,采用传统优化算法求解往往因为计算量巨大而变得不可行,因此需更加高效且智能的优化策略。微服务部署优化涉及的性能指标相当广泛,如 CPU 利用率、内存使用率、网络 I/O 负载、服务响应时间等,这些指标相互制约,难以同时达到最优,因此进一步增加了该问题的复杂性。此外,随着服务的频繁迭代和不断的功能扩展,优化也应具有动态性,能够快速响应结构变化,进行有效的资源重配和负载调整。综上所述,微服务部署优化问题已经成为当前微服务架构软件的研究热点。

1.1.2 研究意义

目前,在微服务部署领域中,最为基本的部署策略为基于规则流程的部署方式。以 Kubernetes (由 Google 开源的微服务容器编排平台) 为例,其默认调度部署策略主要是根据节点的可用 CPU、内存和节点健康状态等信息去进行微服务部署。这种方法具备流程清晰明确的特点,并且部署的速度相对较快。然而,它也存在着一一定的不足之处,即很大程度上忽略了如微服务之间的调用关系、性能方面的约束等重要信息,因此存在部署效果不佳的情况。

为了有效克服这些局限性,众多文章开始针对微服务部署问题进行了深入研究。有研究会将微服务部署问题建模成整数线性规划问题,这种方法能够较为清晰地描述出问题中的各种约束条件和目标,且能够找到其理论精确解。但该方法的计算复杂度极高,尤其是在处理有多个决策变量的大规模问题时,会导致实际求解过程的求解复杂度呈指数增加,可能需要很长时间或者无法在有效时间内搜索到解^[7]。

强化学习也常常应用于在微服务部署领域,其基本原理是可以让机器依据环境所给予的反馈来学习如何做出最为优化的决策^[8]。由于它具有可以实时获取信息来改进策略的特性,使得在这种方法下进行的微服务部署能够较好地适应不断变化的系统资源状态以及服务需求。但他的缺点也很明显,强化学习的需要大量交互数据,训练所耗费的时间较长,收敛速度较慢,因此仍然存在一定的局限性。

近年来启发式算法在微服务部署领域的研究逐渐增多。其在处理复杂的大规

模时,有着能够在有效合理的时间内找到满意解或近似最优解的优点。但是启发式算法在搜索时随机性较强,且优化结果容易受到初始条件与算法参数的影响,进而对结果的稳定性产生不利影响^[9]。不同类型的启发式算法也有着不同的特点,例如遗传算法有较强的全局搜索能力,但是其收敛速度较慢;粒子群算法在实现简单,收敛速度快,但是其容易陷入局部最优解。

遗传算法与粒子群算法是经典的启发式算法,两者在多目标优化领域均有出色的能力,很适用于微服务部署问题。本文拟研究遗传算法与粒子群算法的特点,充分吸收两者的优点,进而设计出一种融合两种算法的双启发式搜索算法,并将其应用在微服务部署领域之中。

1.2 国内外研究现状

对于微服务部署优化问题,林闯等人对于服务计算中优化服务质量的多目标优化方法进行了深入研究,对比分析了不同多目标优化模型对于服务质量计算的影响,为服务系统优化问题提供了新思路^[10]。

传统的数学建模方法大部分将微服务部署问题建模成一个整数线性规划问题(ILP)。如 Guan 等人在 ILP 的基础上引入了一个基于 AODC 的资源分配框架,这是一种面向应用的动态资源分配框架,它解耦了每个应用程序的资源管理和任务执行^[11]。Zhou 等人提出了基于 ILP 和整数二次规划(IQP)方法的在线配置方案,该方案关注集群所需的容器成本和容器间的流量通信,但是为了求解该 NP 难问题,求解时失去了一些准确性^[12]。Rodrigues 等人提出了一种网络感知的多标准容器调度算法来共同处理容器和网络需求^[13]。他们采用混合 ILP 方法来减少数据中心网络碎片和处理延迟,并开发设计了两种 GPU 加速多标准算法来解决该复杂问题。

强化学习是微服务部署问题中的一种常用方法,该方法通过不断收集信息并学习来做出部署决策。例如 Lv 等人建立了基于通信开销和负载均衡的多目标微服务部署问题模型,并提出了基于学习的奖励共享深度 Q-learning 强化学习算法 RSDQL 和基于启发式的弹性缩放算法 ES 来优化微服务部署过程^[14]。张伟颖将容器部署优化问题抽象为一个混合整数优化问题,将上述问题转化为马尔可夫决策过程,提出并改进了基于元-优势演员-评论家的容器化网络功能服务实例动态部署算法^[15]。Wang 等人研究了边缘云之间的微服务协调,以实现移动用户的服务请求的实时响应^[16]。他们提出了一种基于动态规划的离线微服务协调算法,然后利

用马尔可夫决策过程框架对微服务协调问题进行了重构,并提出了一种基于强化学习的在线微服务协调算法来学习最优策略。Chen 等人为了减少混合环境中物联网设备的平均等待时间,提出了一种基于边缘云混合环境中异构和动态特征的基于微服务的部署问题(MSDP)^[17]。为解决该问题,他们设计了一个多缓冲区深度确定性策略梯度(MB-DDPG)来提供更理想的服务部署解决方案,该算法利用强化学习和神经网络来学习部署策略,而无需任何人工指导,服务提供商可以充分利用有限的资源来提高服务质量(QoS)。Xu 等人为优化微服务在不同工作负载下的系统性能和 QoS,提出了一种基于梯度递归单元的预测算法,以准确预测协助缩放的工作负载,并提出了一种称为 CoScal 强化学习的多方面缩放方法来有效地学习缩放技术^[18]。

启发式算法也常被用于解决微服务部署问题。常见的启发式算法有遗传算法 (Genetic Algorithm, GA)、蚁群优化 (Ant Colony Optimization, ACO) 算法和粒子群优化 (Particle Swarm Optimization, PSO) 算法等。启发式算法能够在合理的时间内找到近似最优解,从而可以更好地应用于微服务部署问题。例如 Tan 等人将一种基于双染色体表示的 GA 算法用于求解容器部署问题,以最小化数据中心的能源消耗^[19]。Mao 等人根据物理节点计算与存储资源利用率、微服务请求数量和物理节点故障率,提出了一种基于容器的微服务调度多目标优化模型,并利用 ACO 算法解决该微服务容器调度问题^[20]。陆海锋等人通过考虑微服务之间的调用关系,提出了一种改进的加速粒子群优化算法,用于解决集群中微服务容器部署的多目标优化问题^[21]。Chen 等人提出了一种改进的遗传算法 IGA,通过引入两种不同的交叉变异操作,来优化其模型中的功耗目标^[22]。马武彬等人对基于多目标优化的微服务组合调度问题进行建模,并引入基于参考点非支配排序遗传算法 (NSGA-III) 对该问题进行求解^[23]。Shi 等人提出了一种基于 NSGA-II 的算法来最小化能量消耗和联合容器迁移的数量^[24]。Zuo 等人针对云计算种的任务调度问题,构建了资源成本模型,并通过使用改进的 ACO 算法来优化性能与用户预算成本。这种算法使用了两个约束函数来提供反馈,并根据反馈及调整解的质量,以实现最优解^[25]。Chen 等人提出基于容器的微服务调度的多目标优化问题,结合并行计算和帕累托优化理论,设计了并行粒子群优化算法^[26]。Li 等人提出了研究了基于改进的 NSGA-II 的微服务放置方法 GA-MSP,用于优化移动边缘计算(MEC)问题下的微服务部署^[27]。

由于启发式算法泛用性强的特点，其在这种微服务资源分配问题上已经提出了许多混合启发式算法方案^{[28][29]}。Seghir 等人为解决 QoS 感知的云服务组合问题，提出了一种混合遗传算法 HGA，该算法通过结合遗传算法阶段和果蝇优化阶段来优化搜索过程^[30]。Hussein 等人为提高虚拟机资源利用率，提出了两种调度启发式算法，包括最佳拟合 BF 和最大拟合 MF 算法以及基于 BF 的 ACO 算法，同时使用了适应度函数来评估剩余资源浪费^[31]。Mainak Adhikari 等人采用了加速粒子群算法 APSO，基于加权和方法为每个延迟最小的任务寻找合适的容器，同时又设计了基于规则的策略，以更好地利用服务器的多个资源^[32]。

未来的研究方向聚焦于开发更为先进的资源分配策略，以优化微服务部署过程与效果，并充分利用任务占用资源量与任务间的相关信息，进一步提升微服务架构软件的性能。

1.3 研究内容

本文将深入探讨如何优化微服务部署过程，重点关注如何利用双启发式算法解决多目标的微服务部署优化问题。并对遗传算法与粒子群算法进行融合与改进，以实现更高效可靠的微服务部署决策。最后通过实机部署测试来验证该算法的有效性。

本文的工作内容如下：

首先，本文通过分析微服务的原理特性，构建多目标微服务优化部署模型。该模型以微服务部署约束、集群节点的资源总量约束作为优化问题的约束条件，以最小化微服务网络延时开销和最优化集群负载均衡性能作为优化目标。

其次，本文根据上述多目标优化部署模型，创新式地融合了遗传算法与粒子群算法，设计并实现了一种融合改进的双启发式搜索算法 MOGA-PSO。该算法以 NSGA 和 MOPSO 算法为基础，在进行搜索时根据预设值随机选择 GA 或 PSO 子搜索算法。MOGA-PSO 算法克服了原有遗传算法收敛速度慢、粒子群算法容易是陷入局部最优的问题，吸收了遗传算法全局搜索能力强与粒子群算法收敛速度快的优势，能够有效应用于微服务部署问题中。

最后，本文通过设计实验来对比评估改进算法 MOGA-PSO 算法与 NSGA、MOPSO、Kubernetes 原生策略的性能。本文搭建了由多台虚拟机组成的 Kubernetes 集群，并在上面运行 Bookinfo 微服务应用程序。将 Bookinfo 微服务按照部署结果

运行在集群中后，将利用 Jmeter 软件来测试服务在不同并发请求下的网络延时，利用 Kubernetes 的资源监控系统获取节点资源使用情况来计算集群的资源均衡情况。最后对结果进行了分析与评估，验证了 MOGA-PSO 双启发式算法在微服务部署问题中的有效性。

1.4 论文章节安排

第一章为绪论，介绍了微服务架构与传统单体应用之间的区别，然后引申出微服务的组合部署问题。通过梳理国内外近几年对于微服务自动化部署问题的研究现状，发现其中待改进的问题，总结可以优化改进的方向。明确本文的优化方向与具体工作内容。

第二章为相关理论及概念，介绍了本文所涉及的微服务相关技术和算法理论概念。其主要有 Kubernetes 容器编排平台、Bookinfo 微服务、Jaeger 微服务链路追踪工具、GA 遗传算法与于其对应的多目标搜索算法 NSGA 算法、PSO 粒子群算法与与其对应与的多目标搜索算法 MOPSO 算法、强化学习算法。为接下来介绍本文的微服务部署模型与改进双启发式多目标搜索算法 MOGA-PSO 提供先验知识。

第三章为微服务部署问题与多目标优化模型，本章详细介绍了本文对于微服务部署问题的多目标优化模型的构建过程、构建方式。本章第一节模型定义通过分析微服务使用场景，构建出了基础的数学模型；第二节目标函数通过分析实际部署时需要优化的方面，提出了本模型所致力于优化的微服务网络延时开销和集群负载均衡性能两个目标；第三节约束条件中通过分析实际部署微服务时不合理的部署策略进行了归纳总结，抽象为数学模型中的约束；第四节多目标优化问题数学模型中归纳总结上面三节的设计研究，明确了本文对于微服务部署问题的数学模型，确定了本文算法的研究基础；第五节参数说明中对上述的模型的参数进行了归纳总结。

第四章为基于双启发式搜索的微服务部署算法设计与实现，本章详细描述本文基于 MOGA-PSO 的双启发式微服务部署算法的整体流程与详细设计。整体流程部分通过算法流程图与伪代码展示了算法在遗传算法与粒子群算法融合方面做出的工作。详细设计部分分模块地描述了算法的设计思路与细节。

第五章为实验结果与对比，通过在实际的软硬件环境下，将本文设计的 MOGA-PSO 算法与传统 NSGA 算法、MOPSO 算法与进行实验对比。同时加入

Kubernetes 策略，在实机 Kubernetes 集群中对 Bookinfo 微服务进行了部署测试。算法测试实验结果表明 MOGA-PSO 算法在搜索性能方面比 NSGA、MOPSO 有更高的寻优效率，具体结果为 MOGA-PSO 算法对在收敛速度上相比于 NSGA 优化了 78.5%，在解集质量上较 NSGA 和 MOPSO 算法有了大幅提升。微服务实机部署测试结果表明 MOGA-PSO 算法与 Kubernetes 原生策略相比有一定的优化能力，尤其体现在能有效加快高并发需求下的请求响应时间，具体结果为 MOGA-PSO 算法相较于 Kubernetes 原生策略在高并发请求下的平均延时优化了 18.4%，P95 尾部延时优化了 20.9%。

第六章为总结与展望，将本文的研究成果与实验中发现的问题进行总结归纳，并且提出后续的研究方向和改进。

第二章 相关理论及概念

2.1 多目标优化模型

2.1.1 基本概念

在微服务部署领域，多目标优化是一个非常重要的应用。微服务部署通常需要考虑多个目标，如最小化网络延迟、最大化负载均衡率、最大化资源利用率、最小化能耗等。这些目标往往是相互冲突的，例如，最大化负载均衡率可能会增加延迟，而最小化延迟可能会降低负载均衡率。因此，微服务部署通常需要找到一个平衡不同目标的解决方案，而多目标优化模型可以解决这类问题。

多目标优化问题指的是包含有多个目标函数的优化问题，其数学模型表示方法如下：

$$\begin{aligned}
 & \max/\min f_1(x) \\
 & \max/\min f_2(x) \\
 & \quad \vdots \\
 & \max/\min f_k(x) \\
 & \text{subject to } x \in S
 \end{aligned} \tag{2.1}$$

其中， $\max/\min$ 表示目标函数向着越大还是越小的方向进行优化，例如对网络延时的优化应该是向着最小化，资源利用率应该是向着最大化。为了使模型简介，一般可以将最大化目标函数转换成最小化目标函数，常用的转换方式为取相反数或者取倒数。 $k \geq 2$ 是优化目标函数个数， S 是可行解的搜索空间，也称为解空间，其通常是以约束条件的形式表示的。对于有 p 个小于等于约束， q 个等式约束的多目标优化问题，其有着如下的数学模型：

$$\begin{aligned}
 & \min f_1(x) \\
 & \min f_2(x) \\
 & \quad \vdots \\
 & \min f_k(x) \\
 & \text{s.t. } g_i(x) \leq 0, i = 1, 2, \dots, p \\
 & \quad h_i(x) = 0, i = 1, 2, \dots, q
 \end{aligned} \tag{2.2}$$

对于多目标优化问题，一般没有满足所有目标函数的最优解，几个目标函数都是相互制约相互冲突的，因此对于解决多目标优化模型，有多种思路，例如最经典的对目标值进行线性加权，通过给不同目标函数不同权重值，将多目标问题转换成单目标问题，这样就可以得到单个解，但是其缺点在于会丢失很多信息，同时权重微小变化会对目标值产生很大的影响，因此无法保证解的优劣。帕累托优化是一种

常见的应用于多目标优化问题的方法，这种方法会努力寻找一组解，这些解在所有目标上都不劣于其他解，这些解被称为帕累托最优解，它们代表了不同目标之间的最佳权衡。该方法能够考虑到所有目标和约束，能为决策者提供一组解决方案，因此很适合微服务部署问题。本文所采取的多目标模型即为 Pareto 模型。

2.1.2 帕累托模型

帕累托（Pareto）模型是经典的多目标优化模型。在该模型中，比较两个解的优劣时，有一个重要的概念叫支配。一个解 A 支配另一个解 B 表示解 A 在所有目标上都不劣于解 B，且至少在一个目标上 A 优于 B。当优化方向为最小化目标函数时，对于任意两个可行解 x_1 与 $x_2 \in S$ ，当公式 2.3 成立时，成为 x_1 支配 ($>$) x_2 。该公式含义为， x_1 对应的所有目标函数值都不大于 x_2 的目标函数值，且至少有一个严格小于 x_2 的目标函数值。

$$\begin{aligned} f_i(x_1) &\leq f_i(x_2), \forall i \in \{1, \dots, k\} \\ f_i(x_1) &< f_i(x_2), \exists i \in \{1, \dots, k\} \end{aligned} \quad (2.3)$$

帕累托模型中经常会遇到对于两个解，它们在不同目标上各有优势，因此无法判断哪一个解决方案更优的情况。例如目标函数均为求最小值的情况下，对于 $x_1 = (10, 20)$ 与 $x_2 = (20, 15)$ 两个解， x_1 的第一个目标函数优于 x_2 ， x_2 的第二个目标函数优于 x_1 ，这两个解互相不支配对方。对于一个解来说，在所有的目标上，这个解不被其他任何解所支配，且至少在一个目标上优于其他解，则该解称为帕累托最优解。如果我们能找到一组这样的解的集合，则这个解集称为帕累托前沿。帕累托前沿的数学模型可以表示成公式 2.4 的形式。

$$\{x | \neg \exists x' > x, \forall x' \in S \wedge x' \neq x\} \quad (2.4)$$

在二维目标空间中，帕累托前沿可以被视为一个曲线，所有位于这个曲线上的点都是帕累托最优的。对于更高维度的目标空间，帕累托前沿可以被视为一个曲面或者超平面。在这个帕累托前沿中的解没有好坏之分，每一个解都是代表该问题的最优解，它提供了一种在多个相互冲突的目标之间取得平衡的方式。

2.2 优化算法

2.2.1 遗传算法

遗传算法（Genetic Algorithm，简称 GA）是一种模拟自然界生物进化过程的

计算模型，最早由美国计算机科学家 John H. Holland 于 20 世纪 70 年代提出^[33]。该算法起源于对生物系统所进行的计算机模拟研究，是一种随机全局搜索优化方法，它模拟了自然选择和种群在遗传中所发生的选择、交叉和变异等现象。算法先对解空间进行编码，然后初始化种群，通过随机选择、交叉和变异操作，产生一批适应度函数更高的种群，使群体进化到搜索空间中离目标值越近的区域，这样一代一代不断迭代进化，最后生成一群最适应环境的个体，从而求得问题的近似最优解。

遗传算法的具体过程如下：

步骤 1：初始化：首先，确定问题的解空间，并将解空间中的可能解表示为个体（或称为染色体）。随机生成一个初始种群，这个种群由多个个体（染色体）组成，每个个体代表一个潜在的解决方案。

步骤 2：适应度评估：定义一个适应度函数，该函数能够量化每个染色体的优劣。对种群中的每个染色体进行适应度评估，得到一个适应度值，这个值通常与问题的目标函数相关。

步骤 3：选择：根据每个个体的适应度值，使用某种选择策略（如轮盘赌选择、锦标赛选择等）从当前种群中选择出一部分个体。这些被选中的个体将有机会参与到后续的交叉和变异操作中，以产生新的个体。

步骤 4：交叉：对选择出来的个体进行随机配对。对每一对配对的个体，以一定的交叉概率交换它们之间的部分基因，从而产生两个新的后代个体。交叉操作可以引入新的遗传信息到种群，增加种群的多样性。

步骤 5：变异：对交叉产生的新个体（或整个种群中的一部分个体），以一定的变异概率改变其基因串中的某些基因值。变异操作可以进一步增加种群的多样性，并有助于算法跳脱出局部最优解。

步骤 6：形成新一代种群：将经过交叉和变异操作产生的新个体加入种群中，同时淘汰种群中适应度较低的个体。这样，就形成了一个新一代的种群。

步骤 7：终止条件判断：判断算法是否满足终止条件，如达到最大进化代数、找到满足要求的解等。如果满足终止条件，则算法结束，并输出当前种群中适应度最高的个体作为问题的最优解或近似最优解；否则，返回步骤 2，继续进化过程。

2.2.2 NSGA 算法

对于微服务部署优化问题，通常使用的一种启发式算法是非支配排序遗传算

法 (Nondominated Sorting Genetic Algorithm, 简称 NSGA)。它基于 Pareto 最优概念, 将多目标优化问题中的解集划分为多个非支配层, 每一层中的解在至少一个目标上优于或等于其他层的解, 并通过遗传算法的方式寻找最优的非支配解集。算法主要分为一下几个步骤:

步骤 1: 快速非支配排序: 对初始种群进行非支配排序, 将种群中的解分为不同的非支配层。首先, 找出种群中不被其他任何解支配的解, 并将它们放入第一层。然后, 从剩余的解中找出不被其他剩余解支配的解, 并将它们放入第二层, 以此类推, 直至所有解都被分层。

步骤 2: 选择操作: 根据非支配层进行选择操作。可以用精英保留策略进行选择, 选出非支配排序第一层的解直接进入下一种群, 对其他层的解进行补充选择。

步骤 3: 交叉和变异操作: 对选出的解进行交叉和变异操作, 生成新的解集。交叉操作可以采用单点交叉、多点交叉等方法, 而变异操作可以对解中的某些元素进行随机改变。

步骤 4: 环境选择: 将原始种群和新生成的种群合并, 进行根据适应度函数进行非支配排序计算, 选择合并种群中前 N 个解作为新的种群。

步骤 5: 循环迭代: 重复步骤 1 至步骤 4, 直至满足算法的终止条件 (如最大迭代次数、找到满足要求的解等)

遗传算法在微服务部署问题上应用广泛。由于它只需要适应度函数的信息, 而不需要问题的梯度信息, 因此可以应用于复杂的非线性问题。他有着全局搜索能力强, 不容易陷入局部最优解, 但是其搜索速度慢, 要得到较精确的解需要较多的训练时间。本文的双启发式融合算法参考了遗传算法, 吸收其全局搜索能力与在处理非线性问题上的优势, 有针对性的适应微服务部署的具体应用场景, 旨在更有效地解决微服务部署中的优化问题。

2.2.3 粒子群算法

粒子群算法, 全称为粒子群优化算法 (Particle Swarm Optimization, 简称 PSO), 也是一种随机全局搜索优化方法, 其基本原理源于对鸟群或鱼群等群体行为的模拟^[34]。在这个算法中, 每个个体被视作一个“粒子”, 代表问题空间中的一个潜在解。这些粒子在搜索空间中共享信息并协作, 并根据信息去更新自身的速度与位置, 以寻找最优解。由于其实现简单、收敛速度快、全局搜索能力强等特点, 粒子群算法

在函数优化、模糊系统控制等领域得到了广泛应用。

在粒子群算法中，每个粒子都代表问题的一个潜在解，并具有位置、速度和适应度值三个属性。位置表示粒子在搜索空间中的位置，速度表示粒子飞行的方向和快慢，适应度值则用于评价粒子的优劣。算法通过迭代更新粒子的位置和速度来寻找最优解。

粒子群算法的重要公式为速度更新公式和位置更新公式。速度更新公式考虑了粒子自身的速度惯性、个体历史最优位置的影响和群体最优位置的影响。位置更新公式则根据更新后的速度计算粒子的新位置。

具体来说，假设粒子群中有 N 个粒子，第 i 个粒子的位置为 x_i ，速度为 v_i ，个体最优位置为 $pbest_i$ ，群体最优位置为 $gbest$ 。在每次迭代第 t 轮中，粒子的速度和位置按以下公式进行更新：

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t)) \quad (2.5)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (2.6)$$

速度更新公式为公式 2.5，其中， w 为惯性权重，用于控制粒子速度的惯性大小； c_1 和 c_2 为速度权重因子，分别表示粒子朝向个体最优位置和群体最优位置前进的权重； r_1 和 r_2 为随机数，用于增加算法的随机性。位置更新公式为公式 2.6，该公式表示粒子通过矢量相加的方式利用更新后的速度移动到新的位置。粒子群算法流程如下：

步骤 1：初始化粒子群：定义问题的搜索空间，即解空间。随机生成一定数量的粒子，每个粒子代表搜索空间中的一个解。为每个粒子随机初始化位置和速度，并为每个粒子设置个体最优位置，初始值设为该粒子的初始位置。初始化全局最优位置，通常从所有粒子的初始位置中选择适应度最好的一个作为初始全局最优位置。

步骤 2：计算适应度：对于每个粒子，计算其当前位置的适应度值。适应度函数是根据问题的目标函数来定义的，用于评估粒子位置的优劣。

步骤 3：更新个体最优位置和全局最优位置：比较每个粒子的当前适应度值与其个体最优位置的适应度值，如果当前适应度值更优，则更新该粒子的个体最优位置为当前位置。比较所有粒子的个体最优位置的适应度值，找出适应度最优的个体最优位置，将其作为当前的全局最优位置。

步骤 4: 更新粒子的速度和位置: 根据粒子的当前速度、个体最优位置和全局最优位置来更新粒子的速度和位置, 参考公式 2.1。粒子的位置更新则是基于更新后的速度进行的, 参考公式 2.2。

步骤 5: 终止条件判断: 判断算法是否满足终止条件, 如达到预设的最大迭代次数、适应度值满足一定要求等。如果满足终止条件, 则算法结束, 并输出当前种群中适应度最高的个体作为问题的最优解或近似最优解; 否则, 返回步骤 2, 继续搜索过程。

2.2.4 MOPSO 算法

MOPSO 算法, 全称为多目标粒子群优化算法 (Multi-Objective Particle Swarm Optimization), 也常常用于求解多目标优化问题。MOPSO 算法与传统的单目标 PSO 算法相比, 在优化过程中考虑了多个目标, 采用 Pareto 支配关系来评价个体的优劣, 并利用外部仓库 (Archive) 来存储算法执行过程中的非支配解。MOPSO 算法的主要步骤如下:

步骤 1: 初始化: 设置算法参数, 如种群大小、最大迭代次数、惯性权重等。随机生成一定数量的粒子, 每个粒子代表问题的一个潜在解。为每个粒子分配初始位置和速度, 并初始化外部仓库为空。

步骤 2: 评价粒子并更新外部仓库: 计算每个粒子的适应度值, 对于多目标问题, 需要计算各个目标的适应度值。通过对粒子群进行非支配排序, 将非支配解添加到外部仓库中, 同时根据 Pareto 支配关系和外部仓库的容量限制, 对仓库中的解进行筛选, 保留优良的非支配解, 更新外部仓库。

步骤 3: 选择全局最优解: 从外部仓库中根据某种选择策略 (如随机选择、拥挤距离等) 选择一个全局最优解, 用于更新粒子的速度和位置。

步骤 4: 更新粒子速度和位置: 根据粒子自身的历史最优位置 (个体最优解) 和群体最优位置 (全局最优解) 来更新粒子的速度和位置。

步骤 5: 循环迭代: 重复步骤 2 至步骤 4, 直至满足算法的终止条件 (如最大迭代次数、外部仓库的收敛性等)。

粒子群算法虽在优化性能上表现出色, 但会面临着算法在未达到全局最优解时便提前收敛的问题。为了克服这一挑战, 许多研究者对该算法进行了深入的改进与调整。本文的双启发式融合算法借鉴参考粒子群算法的思想, 在微服务部署的具

体应用场景上对算法进行了针对性的适配和改良，进一步提高算法在解决微服务部署问题的能力。

2.2.5 强化学习算法

强化学习（Reinforcement Learning）是机器学习领域中的一种重要方法，经常被用于微服务部署问题上^[35]。它以“趋利避害”为原则，旨在通过智能决策体（Agent）与环境（Environment）的持续交互来学习和优化行为策略。该过程的核心在于，智能体通过学习当前环境状态来选择最优的动作，以最大化长期累积的奖励信号（Reward）。强化学习的工作模式基于智能体不断探索与利用已有经验之间的平衡，通过不断反馈调整，逐步逼近最优的行为策略。图 2.1 为强化学习工作模式的基本结构。

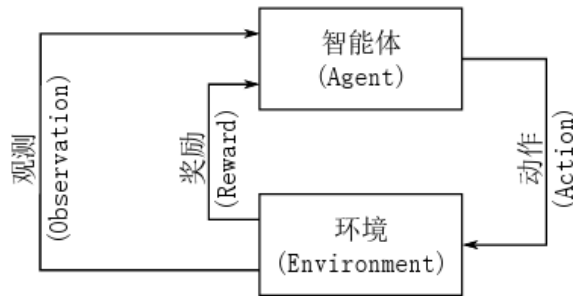


图 2.1 强化学习基本结构

在实际应用中，可以通过以下步骤将强化学习应用于微服务部署问题：

步骤 1：定义问题与奖励函数：首先，需要明确微服务部署的目标和要求，如服务响应时间、集群负载均衡、成本等。根据这些目标来定义奖励函数，用于评估不同部署策略的效果。

步骤 2：收集数据与环境建模：收集微服务部署的历史数据和环境信息，如服务之间的依赖关系、集群各类资源状况、网络延时开销等。根据这些数据建立环境模型，用于模拟和预测不同部署策略的效果。

步骤 3：训练强化学习模型：使用收集的数据和环境模型来训练强化学习模型。在训练过程中，模型会尝试不同的部署策略，并根据奖励函数的反馈进行调整和优化。

步骤 4：部署与监控：将训练好的强化学习模型应用于实际的微服务部署中。在部署过程中，模型会根据实时的环境信息和业务需求进行自动决策和调整。同时，通过监控系统的运行状态和性能指标，可以及时发现并解决问题。

步骤 5: 迭代与优化: 在微服务部署的过程中, 不断地收集新的数据和反馈, 用于更新和优化强化学习模型。通过迭代和优化, 可以不断提高微服务部署的效率和质量。

2.3 微服务相关软件

2.3.1 Kubernetes 容器编排系统

Kubernetes 是业界主流使用的一种微服务编排平台系统, 可用于自动部署、伸缩和管理容器化应用程序^[36]。Kubernetes 集群的架构中至少具有一个主节点(master)和若干个工作节点(node), 在这些节点上会运行容器化应用程序。Kubernetes 在中, pod 是最小的可部署单元, pod 内可以包含一个或多个容器, 这些容器共享相同的网络和存储资源。在每个节点上会运行相应的节点组件, 负责维护运行的 pod 并提供 Kubernetes 运行环境。kubelet 组件是集群中每个节点上运行的代理, 保证容器都运行在 pod 中。其体系结构如图 2.2 所示。

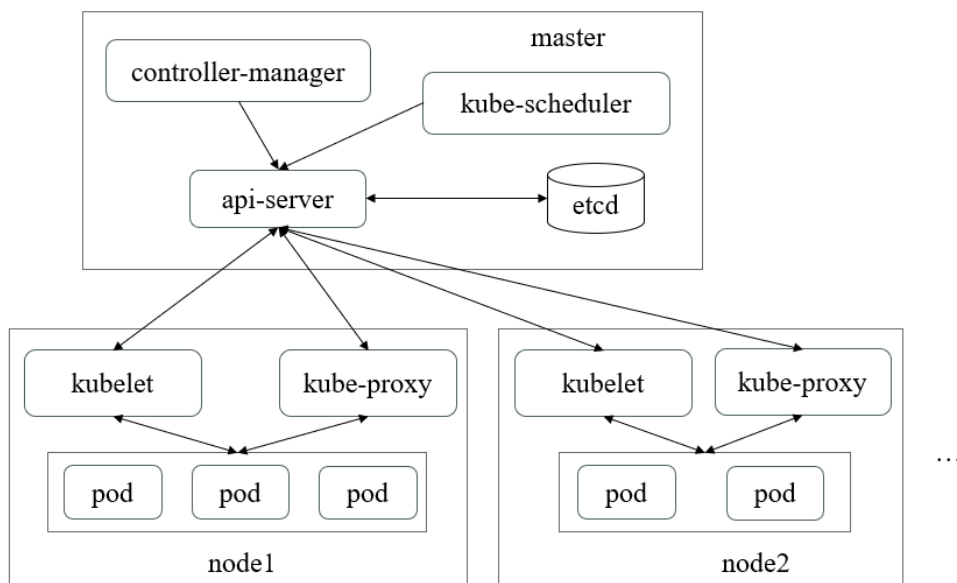


图 2.2 Kubernetes 集群架构

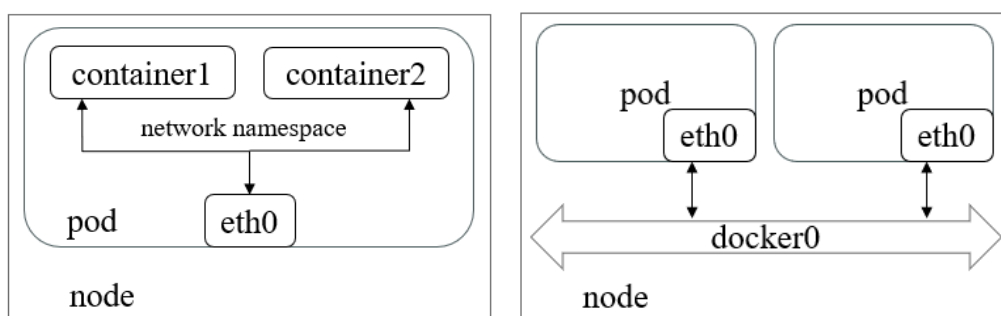
pod 之间的通信在 Kubernetes 集群中的形式主要有: 同一 pod 内容器通信、同一 node 不同 pod 通信、不同 node 上 pod 通信三种情况。

对于在同一 pod 内的容器可以直接通过 localhost 进行通信。如图 2.3(a), 它们共享同一个网络命名空间, 无需经过网络层, 其具有最低的延迟。

同一 node 不同 pod 之间的通信可以直接通过 docker0 网桥进行。如图 2.3(b), docker0 网桥会负责在不同 pod 之间转发数据包, 因为它们都在同一个网桥下的不

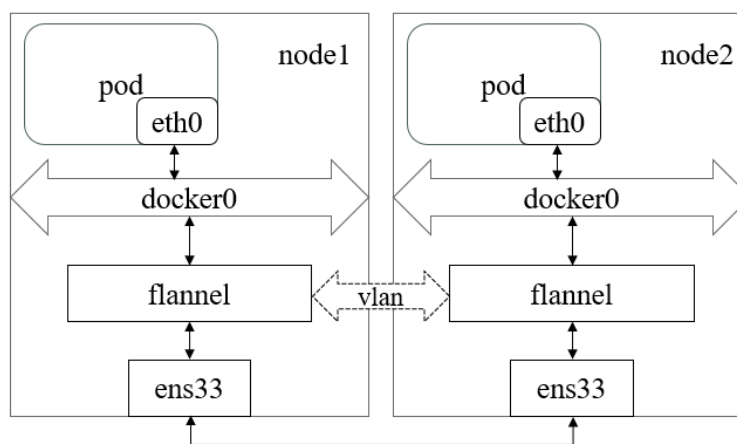
同子网中。

对于不同 node 上 pod 通信的情况相对复杂，需要通过集群网络插件来实现。这些网络插件负责在不同节点之间建立网络连接，使得 pod 可以跨节点通信。如图 2.3(c)，在 docker0 网桥外还有一层容器网络接口 CNI 插件（如 flannel 等）配置的网络隧道。新的虚拟网卡 flannel 接收 docker0 网桥的数据，并通过维护路由表，对接收到的数据进行封包和转发。flannel 通过节点网卡 ens33 与另一个节点进行通信，实现虚拟局域网。



(a) 同一 pod 内容器间通信

(b) 同一 node 内 pod 间通信



(c) 不同 node 间 pod 通信

图 2.3 Kubernetes 集群中容器间通信

在 Kubernetes 中，kube-scheduler 组件是集群的默认调度器，也是集群控制层的一部分，它的核心功能是负责 pod 调度部署。在整个 pod 的部署环节，kube-scheduler 根据特定的调度算法和策略，将待部署的 pod 部署到集群中合适的 node 上，并将信息写入 etcd 中。kube-scheduler 在调度部署 pod 时包含两个步骤：

1、过滤

过滤阶段，kube-scheduler 会选出所有满足 pod 调度需求的 node。例如，

PodMatchNodeSelector 过滤函数会匹配 pod 的 nodeSelector 与 node 的标签(label), 然后在 pod 的配置列表中加入含有指定标签的 node。在过滤完成时, kube-scheduler 会得到一个包含了该 pod 所有可调度 node 列表。若该列表为空, 则代表该 pod 不可调度。Kubernetes 调度器的过滤规则见表 2.1。

表 2.1 微服务部署过滤规则

名称	功能
PodFitsHostPorts	检查 node 上是否存在当前被调度 pod 的端口(如果被调度 pod 用的端口已被占用, 则此 node 被 pass)。
PodFitsHost	检查 pod 是否通过主机名指定了特定的 node(是否在 pod 中定义了 nodeName)。
PodFitsResources	检查 node 是否有空闲资源(如 CPU 和内存)以满足 pod 的需求。
PodMatchNodeSelector	检查 pod 是否通过节点选择器选择了特定的 node (是否在 pod 中定义了 nodeSelector)。
NoVolumeZoneConflict	检查 pod 请求的卷在 node 上是否可用(不可用的 node 被 pass)。
NoDiskConflict	根据 pod 请求的卷和已挂载的卷, 检查 pod 是否合适于某个 node(例如 pod 要挂载/data 到容器中, node 上/data/已经被其它 pod 挂载, 那么此 pod 则不适合此 node)。
MaxCSIVolumeCount	决定应该附加多少 CSI 卷, 以及是否超过了配置的限制。
CheckNodeMemoryPressure	对于内存有压力的 node, 则不会被调度 pod。
CheckNodePIDPressure	对于进程 id 不足的 node, 则不会调度 pod。
CheckNodeDiskPressure	对于磁盘存储已满或者接近满的 node, 则不会调度 pod。
CheckNodeCondition	node 报告给 api server 说自己文件系统不足, 网络有写问题或者 kubelet 还没有准备好运行 pods 等问题, 则不会调度 pod。

PodToleratesNodeTaints	检查 pod 的容忍度是否能承受被打上污点的 node。
CheckVolumeBinding	根据一个 pod 并发流量来评估它是否合适（这适用于结合型和非结合型 pvcs）。

2、打分

在打分阶段，kube-scheduler 会为 pod 从可调度 node 列表中选出一个最合适的 node。根据配置文件所启用的打分规则，kube-scheduler 会给每一个可调度的 node 进行打分，然后将 pod 调度到得分最高的 node 上。如果存在多个得分最高的 node，kube-scheduler 会从中随机选取一个。Kubernetes 调度器的打分规则见表 2.2。

表 2.2 微服务部署打分规则

名称	功能
SelectorSpreadPriority	优先减少节点上属于同一个 service 或 replication controller 的 pod 数量。
InterPodAffinityPriority	优先将 pod 调度到相同的拓扑上（如同一个节点、rack、zone 等）。
LeastRequestedPriority	节点上放置的 pod 越多，这些 pod 使用的资源越多，这个 node 给出的打分就越低，所以优先调度到 pod 少及资源使用少的节点上。
MostRequestedPriority	尽量调度到已经使用过的 node 上，将把计划的 pods 放到运行整个工作负载所需的最小节点数量上。
RequestedToCapacityRatio Priority	使用默认资源评分函数形状创建基于 requestedtocapacit 的 resourceallocationpriority
BalancedResourceAllocation	优先平衡各节点的资源使用。
NodePreferAvoidPodsPriority	根据节点注释对节点进行优先级排序，以使用它来提示两个不同的 pod 不应在同一节点上运行。
NodeAffinityPriority	优先调度到匹配 nodeaffinity（node 亲和性调度）的节点上。

TaintTolerationPriority	优先调度到匹配 tainttoleration(污点)的节点上。
ImageLocalityPriority	尽量将使用大镜像的容器调度到已经下拉了该镜像的节点上。
ServiceSpreadingPriority	尽量将同一个 service 的 pod 分布到不同节点上，服务对单个节点故障更具弹性。
EqualPriority	将所有节点的权重设置为 1。
EvenPodsSpreadPriority	实现首选 pod 拓扑扩展约束。

本文中的 Kubernetes 默认策略，是通过不指定 pod 部署节点，仅根据节点资源、微服务所需资源等信息，经过上述 Kubernetes 调度器过滤和打分策略进行部署。对于本文双启发式算法搜索到的部署策略解的部署方式，是在 pod 配置文件中定义 nodeSelector 字段，将解写入该字段中来选择指定 node，利用过滤规则中 PodMatchNodeSelector 规则滤除其他节点，经过打分阶段后即可部署在指定 node 上。

2.3.2 Bookinfo 微服务应用

Bookinfo 应用是 Istio 官方提供的用来展示其特性的一个应用^[37]。这个应用模仿在线书店的一个分类，显示一本书的信息。页面上会显示一本书的描述，书籍的细节（ISBN、页数等），以及关于这本书的一些评论。Bookinfo 应用分为四个单独的微服务：

productpage: 该微服务会调用 details 和 reviews 两个微服务，用来生成页面。

details: 该微服务中包含了书籍的信息。

reviews: 该微服务中包含了书籍相关的评论。它还会调用 ratings 微服务。

reviews 微服务有 3 个版本：

v1 版本不会调用 ratings 服务。

v2 版本会调用 ratings 服务，并使用 1 到 5 个黑色星形图标来显示评分信息。

v3 版本会调用 ratings 服务，并使用 1 到 5 个红色星形图标来显示评分信息。

ratings: 该微服务中包含了由书籍评价组成的评级信息。

图 2.4 展示了 Bookinfo 微服务应用的端到端架构。

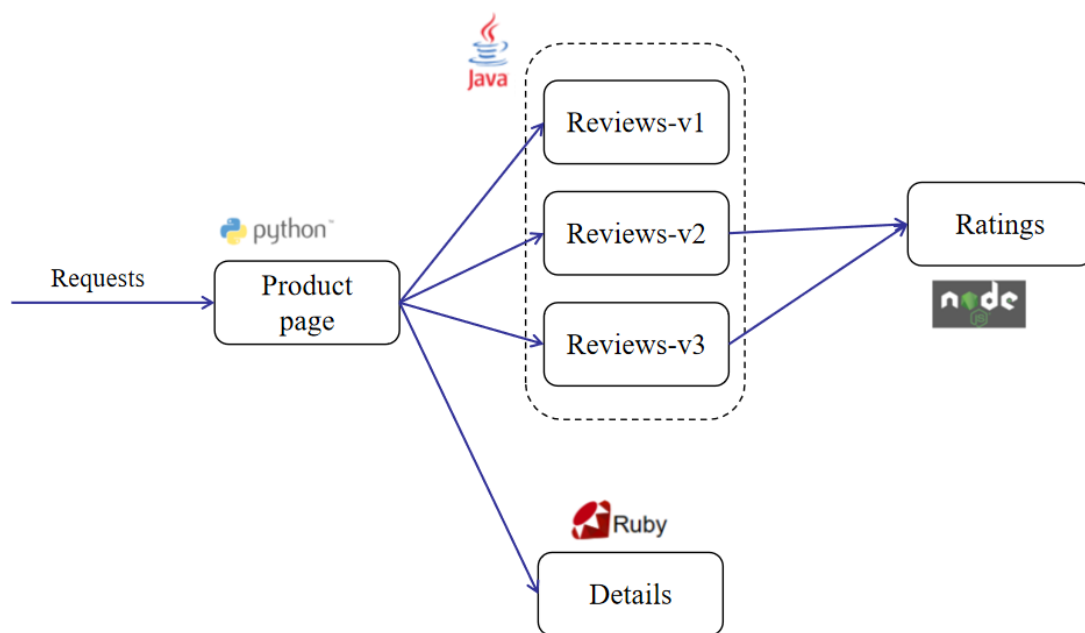


图 2.4 Bookinfo 微服务

本文选用 Bookinfo 微服务来进行实机部署测试，详见第五章。在集群中部署 Bookinfo 微服务利用到了 Istio 中的服务网格特性，需要配置 Envoy Sidecar 注入到每个服务之中，之后每个服务通过与本服务 envoy 组件交互，本服务 envoy 再与其他服务的 envoy 交互实现服务之间的通信。配置完对外的网关后，即可从虚拟机外部通过 ip 地址访问 Bookinfo 页面。对该应用发起请求所返回页面如图 2.5，通过右侧的红色星形图标评分可以知道本次请求使用的是 reviews 的 v3 版本。

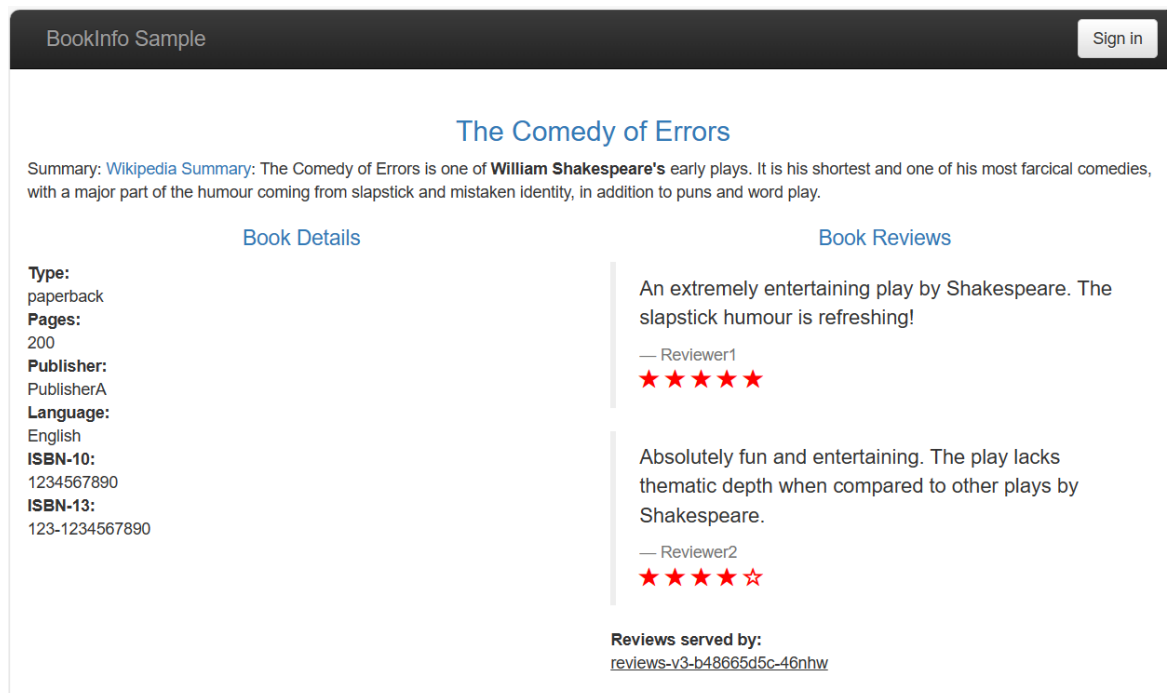


图 2.5 Bookinfo 网页

2.3.3 Jaeger 微服务链路追踪

为了知道一个微服务之间的调用关系以及内部是如何进行调用的，需要微服务链路追踪对微服务进行监控和信息收集。微服务链路追踪通过跟踪微服务之间的调用关系，记录每个请求从发起到结束的完整路线，以及服务之间的时间和延迟信息。微服务链路追踪的工作原理主要包括拦截请求、生成 trace ID、记录调用关系以及存储请求元数据等步骤。通过这种方式，开发人员能够理解和分析分布式系统中的数据流动和性能瓶颈，从而优化系统性能并提升用户体验。

Jaeger 是一个开源的分布式跟踪系统，遵循 OpenTracing 开放式分布式追踪规范，其架构如图 2.6 所示^[38]。它主要用于监控复杂的分布式系统，尤其是由微服务架构或云原生架构所构建的软件系统。Jaeger 的设计目标是提供轻量高效的分布式追踪，同时提供可扩展性。它还提供了一个简单易操作的 UI 界面，使得开发人员和运维人员能够方便地查看和分析追踪数据。

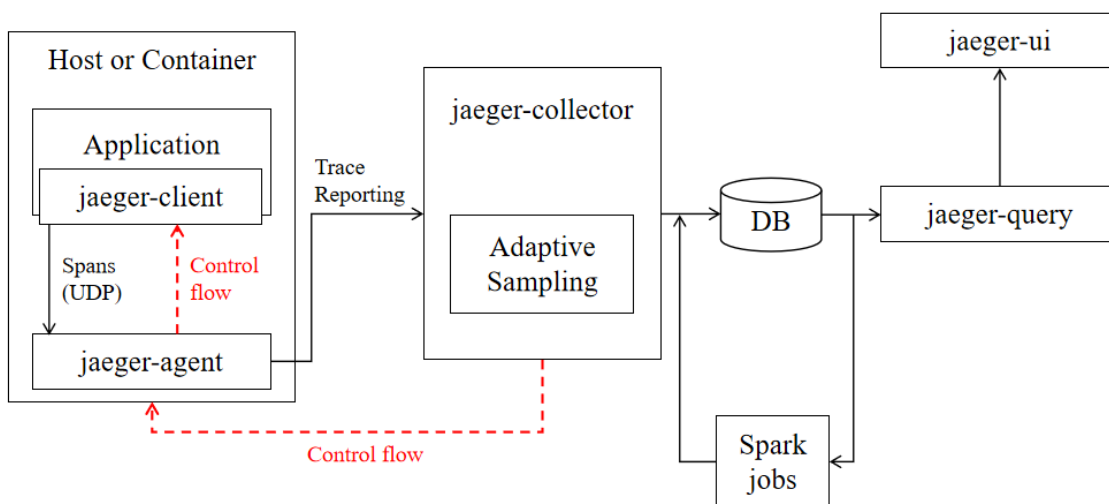


图 2.6 Jaeger 架构

本文中利用 Jaeger 来获取微服务之间的调用关系，监控微服务间的调用延时。如图 2.7 是 Jaeger 监控 Bookinfo 微服务应用所抓取的网络数据。



图 2.7 Jaeger 分布式跟踪

第三章 基于多目标优化的微服务部署模型

3.1 基本定义

首先需要明确本文的研究对象与研究目标。本文的研究对象为特定场景与目标下的微服务部署问题。本文的研究目标为探究微服务部署问题中不同部署策略对微服务网络延时开销与集群负载均衡性能率的影响，根据本文的双启发式优化算法来优化微服务部署过程。为了方便阐述该问题，本文构建了针对微服务部署策略的多目标优化模型，首先明确以下定义。

1、对节点的定义：节点指可以部署微服务实例的计算机，为微服务实例的运行提供环境与资源。微服务节点用 n_i 表示， n_i 节点上有计算资源 $R_{cpu}(n_i)$ 和内存资源 $R_{mem}(n_i)$ ， R_{cpu} 为核心数， R_{mem} 为内存总量。

2、对节点集群的定义：节点集群是多个能够运行微服务实例的节点的集合，这些节点通过网络互相通信协作，来完成一个微服务的运行。节点集群用 N 表示，则一个有 d 个可以运行微服务的节点的集群为

$$N = \{n_1, n_2, \dots, n_d\} \quad (3.1)$$

集群内节点之间通过网络进行通信，通信存在延时。假设 i 节点与 j 节点之间的延时函数为 $ND(n_i, n_j)$ ，相同节点内通信无延时，对含有 d 个节点的集群 N ，节点间通信延时矩阵用 D 表示，则 D 为

$$D = \begin{bmatrix} 0 & ND(n_1, n_2) & \dots & ND(n_1, n_d) \\ ND(n_2, n_1) & 0 & \dots & \dots \\ \dots & \dots & \dots & ND(n_{d-1}, n_d) \\ ND(n_d, n_1) & \dots & ND(n_d, n_{d-1}) & 0 \end{bmatrix} \quad (3.2)$$

3、对微服务的定义：微服务是众多松散耦合且可单独部署的小型组件或服务。微服务用表示 ms_i ，一个微服务实例部署到节点上需要消耗一定量的资源，本文定义微服务 ms_i 消耗的计算资源为 $MSR_{cpu}(ms_i)$ ，内存资源为 $MSR_{mem}(ms_i)$ 。

4、对微服务集合的定义：微服务集合是指可以提供一种或多种完整服务的一组微服务。微服务集合用 MSS 表示，则一个有 m 个微服务的微服务集合为

$$MS = \{ms_1, ms_2, \dots, ms_m\} \quad (3.3)$$

在微服务集合中的，两个微服务之间存在调用关系，即调起一个微服务可能会

连带着调起另一个微服务。本文用调用图 $MSG = \langle MS, MSC \rangle$ 表示微服务之间的调用关系。其中 MS 代表调用图中的顶点集合，含义为微服务集合； MSC 代表调用图中的边集，含义为两个微服务之间是否存在调用关系。

用户的单次请求或操作可能会触发微服务集合中部分微服务按特定顺序进行调起，这形成了微服务调用链。本文定义微服务调用链为 L ，其定义如下：

$$L = \{(ms_i, ms_j), (ms_j, ms_k), \dots\}, \forall (ms_i, ms_j) \in MSC \quad (3.4)$$

上述公式表示在调用链 L 中，微服务 ms_i 为调用链入口， ms_i 会调用 ms_j ， ms_j 会调用 ms_k ，以此类推，直到调用完毕。

不同请求可能会从不同微服务入口进入，可能产生不同的调用链；某些微服务在接受请求后也会根据接收参数不同等因素调起不同的微服务，因此一个微服务集合所提供的服务中常常包含多条微服务调用链。本文定义，若一个微服务集合内包含 l 条调用链，全部调用链的集合用 $MSCL$ 表示，其定义如下：

$$MSCL = \{L_1, L_2, \dots, L_l\} \quad (3.5)$$

5、对微服务部署策略的定义：微服务部署策略指的是将微服务集合中的微服务部署在集群节点的方式。微服务实例会被部署在节点上，因此本文定义，在有 k 个微服务的微服务集合 MS 中，微服务 ms_i 所被部署的节点为 sn_i ， $sn_i \in N$ 。用 DS 表示一个部署策略，则对于节点集群 N ，微服务集合 MS ，一个部署策略可以表示为：

$$DS(N, MS) = \{sn_1^i, sn_2^j, \dots, sn_m^k\} \quad (3.6)$$

sn_1^i 表示微服务 ms_1 被部署到了节点 n_i 上。本文将用向量 $x = DS(N, MS)$ 表示该多目标优化问题的自变量， Ω 表示部署策略向量的解空间，有 $x \in \Omega$ 。

3.2 目标函数

3.2.1 微服务网络延时开销

本文微服务网络延时开销指的是，从用户请求到达微服务入口开始，通过微服务调用链调起本次请求所需的全部微服务所需的时间。一个微服务集合中可能包含多条微服务请求，用户的一次请求会触发调起某一条调用链上的全部微服务。每条调用链所产生的网络延时开销不同，且在多次请求下，每条调用链会按一定概率被使用。因此本文利用网络延迟产生的平均开销作为微服务网络延时开销，用以衡

量大量并发请求所带来的平均延时。

设部署决策为 $\mathbf{x} = DS(N, MS)$ ，节点通信延时矩阵为 D ，每条调用链使用占比为 w_l ，微服务调用链集合为 $MSCL$ 。则微服务调用链 L 所产生的网络延时开销 $netDelayConsume$ 见公式 3.7，其含义为计算了在调用链 L 中连通的全部微服务之间的通信延时之和。平均微服务网络延时开销 $ANDC$ 见公式 3.8，其含义是根据每条调用链的使用占比，计算出全部调用链的平均网络延迟。

$$netDelayConsume(L) = \sum_{\forall (ms_i, ms_j) \in L} D(sn_i, sn_j), sn_i, sn_j \in \mathbf{x} \quad (3.7)$$

$$ANDC(MSCL) = \frac{\sum_{L \in MSCL} w_L \cdot netDelayConsume(L)}{|MSCL|} \quad (3.8)$$

3.2.2 集群负载均衡性能

微服务集合在部署到节点集群后，节点集群中各个节点会消耗相应量的资源。如果某一节点上消耗资源量过多，其余节点资源消耗量少，该集群的总体资源使用量就会下降，服务性能仍会有提升空间。若集群中资源使用量更加均匀地分布在各个节点上，就可以提高系统整体资源利用率，从而提供更优质的服务。因此本文模型的第二个目标即为集群负载均衡性能，该指标可以用来衡量微服务是否均匀地被分配在各个节点上。在该模型中，集群负载均衡性能用集群节点资源利用率的标准差来表示，该值越小表示集群负载均衡性能越优。

在部署策略为 $\mathbf{x}$ 的情况下，节点 n_i 上的所部署的微服务实例集合为：

$$MSON(n_i) = \{ms_j | \forall (ms_j \in MS \wedge sn_j^i = n_i), sn_j^i \in \mathbf{x}\} \quad (3.9)$$

每个节点的资源利用率通过计算该节点上微服务所消耗的每种资源总和比上节点资源量得到，如下分别为 CPU 利用率 $CPUUtilization$ （公式 3.10）、内存利用率 $MEMUtilization$ （公式 3.11）：

$$CPUUtilization(n_i) = \frac{\sum_{ms_j \in MSON(n_i)} MSR_{cpu}(ms_j)}{R_{cpu}(n_i)} \quad (3.10)$$

$$MEMUtilization(n_i) = \frac{\sum_{ms_j \in MSON(n_i)} MSR_{mem}(ms_j)}{R_{cpu}(n_i)} \quad (3.11)$$

标准差能反映一个数据集的离散程度，也可以反应一个数据集是否均衡。因此，本文利用节点间资源利用率的标准差来表示集群负载均衡率，该值越小表示负载

越均衡, 集群负载均衡性能越高。如下分别为 CPU 利用率均值 $CPUUsageRate$ (公式 3.12)、CPU 利用率标准差 $CPUURstd$ (公式 3.13)、内存利用率均值 $MEMURavg$ (公式 3.14)、内存利用率标准差 $MEMURstd$ (公式 3.15) :

$$CPUURavg = \frac{\sum_{n_i \in N} CPUUtilization(n_i)}{|N|} \quad (3.12)$$

$$CPUURstd = \sqrt{\frac{\sum_{n_i \in N} (CPUUtilization(n_i) - CPUURavg)^2}{|N| - 1}} \quad (3.13)$$

$$MEMURavg = \frac{\sum_{n_i \in N} MEMUtilization(n_i)}{|N|} \quad (3.14)$$

$$MEMURstd = \sqrt{\frac{\sum_{n_i \in N} (MEMUtilization(n_i) - MEMURavg)^2}{|N| - 1}} \quad (3.15)$$

对于不同的软件, 对 CPU 和内存的负载均衡性能需求可能有所不同, 因此本文利用线性加权求和法对 CPU 利用率标准差与内存利用率标准差加权求和计算集群负载均衡率, 用户可以通过设置 w_{cpu} 与 w_{mem} 权重来调节 CPU 资源与内存资源对集群负载均衡率中的重要性。集群负载均衡率 $CLBR$ 的公式如 3.16 所示:

$$CLBR(x) = w_{cpu} \cdot CPUURstd + w_{mem} \cdot MEMURstd \quad (3.16)$$

3.3 约束条件

在微服务部署问题中, 解空间全集中的一些部署策略无法被部署到实际集群中, 因此在问题建模中需要对部署策略的解空间进行约束。本文对解空间有如下约束:

1、微服务部署约束: 须保证微服务集合 MS 中每个微服务部署在了集群 N 的节点内, 用公式描述如下:

$$g1: sn_i \in N, 1 \leq i \leq |MS| \quad (3.17)$$

2、计算资源总量约束: 一个节点上运行全部微服务实例所占用的计算资源总量不能超过该节点所拥有的计算资源总量, 用公式描述为:

$$g2: \forall n \in N, R_{cpu}(n) > \left(\sum_{\forall ms_i \in MSON(n)} MSR_{cpu}(n) \right) \quad (3.18)$$

3、内存资源总量约束：一个节点上运行全部微服务实例所占用的内存资源总量不能超过该节点所拥有的内存资源总量，用公式描述为：

$$g3: \forall n \in N, R_{mem}(n) > \left(\sum_{\forall ms_i \in MSON(n)} MSR_{mem}(n) \right) \quad (3.19)$$

3.4 模型构建

综上所述，基于微服务网络延时开销与集群负载均衡性能为优化目标的微服务部署数学模型描述如下：

假设有节点集群 N 与微服务集合 MS ，对于部署策略向量 $\mathbf{x}$ ，则

$$\begin{cases} \min \text{ ANDC}(\mathbf{x}, MSCL) \\ \min \text{ CLBR}(\mathbf{x}) \\ s.t. \\ g1: sn_i \in N, 1 \leq i \leq |MS| \\ g2: \forall n \in N, R_{cpu}(n) > \left(\sum_{\forall ms_i \in MSON(n)} MSR_{cpu}(n) \right) \\ g3: \forall n \in N, R_{mem}(n) > \left(\sum_{\forall ms_i \in MSON(n)} MSR_{mem}(n) \right) \end{cases}$$

微服务部署问题实际上就是在给定的节点集群、微服务集合的前提下，寻找微服务实例在各节点的启动和运行策略，使得在同时考虑节点之间的网络通信与节点集群的负载均衡的情况下，寻找出最优解集。

该模型是一个典型的 NP 难问题，采用传统多项式解决方法难以解决该问题。因此需要采用启发式算法来解决该问题。本文第四章将详细描述如何利用创新融合的双启发式算法 MOGA-PSO 来解决该模型问题。

3.5 参数说明

模型相关参数汇总如表 3.1:

表 3.1 模型参数

名称	符号表示
微服务节点	n_i
节点计算资源	R_{cpu}
节点内存资源	R_{mem}
节点集群	N
节点间延时函数	ND
节点间延时矩阵	D
微服务消耗计算资源	MSR_{cpu}
微服务消耗内存资源	MSR_{mem}
微服务集合	MS
微服务调用关系	MSC
微服务调用链集合	$MSCL$
部署策略向量	DS
部署策略向量的解空间	Ω
调用链使用权重	w_L
微服务网络延时	$netDelayConsume$
平均微服务网络延时	$ANDC$
节点部署微服务集合	$MSON$
节点 CPU 利用率	$CPUUtilization$
节点 MEM 利用率	$MEMUtilization$
集群 CPU 利用率均值	$CPUURavg$
集群 MEM 利用率均值	$MEMURavg$
集群 CPU 利用率标准差	$CPUURstd$
集群 MEM 利用率标准差	$MEMURstd$
集群负载均衡率	$CLBR$

第四章 基于双启发式优化的微服务部署算法设计

4.1 算法整体流程

为了解决第三章所描述的微服务部署问题，本文将利用启发式算法来对该多目标问题进行研究。经典的遗传算法有快速随机的全局搜索能力，但是存在优化过程可能较长，效率不高的问题。粒子群算法有收敛速度快，算法简单易实现的优点，但是有在搜索过程中因搜索范围不够广而导致的容易陷入局部最优的问题。在实际部署中的体现为遗传算法可以找到更优质的解集，但是时间较长，收敛较慢；粒子群算法可以快速收敛到一个稳定的解集中，但是该解集中的解不是全局最优，仍有优化空间。

本文为了解决上述问题，设计了针对多目标优化的双启发式算法 MOGA-PSO，该算法结合遗传算法与粒子群算法的优点，吸收遗传算法快速随机的全局搜索能力，又吸收了粒子群中的粒子共享最优解信息从而加快收敛的优点，以提升搜索算法的性能，优化微服务部署问题。MOGA-PSO 算法总体流程主要经过下面这些步骤：

步骤 1：根据编码规则初始化：初始化粒子群（种群），即随机生成一定数量的粒子，每个粒子代表问题的一个微服务部署策略解，粒子形式按照 4.2.1 编码规则生成。该粒子参与后续的遗传算法与粒子群算法步骤，因此为每个粒子初始化随机位置和速度。初始化最优解外部仓库为空，该最优解外部仓库用于存储当前代数的所有最优解，相当于当前代的帕累托前沿，该解集在算法迭代结束后输出。

步骤 2：根据适应度函数评价粒子：计算每个粒子的适应度值，由于本文为多目标优化问题，因此该适应度函数计算了网络延时开销和集群负载均衡。

步骤 3：更新最优解外部仓库：根据 Pareto 支配关系，将非支配解添加到最优解外部仓库中；同时需要对外部仓库内的解进行非支配排序，去除支配序列中非第一层的解，确保外部仓库内的解均为非支配解。

步骤 4：选择子搜索算法：接下来将会根据规则进行选择下一步是进行遗传算法还是进行粒子群算法。本文采用根据概率选择子搜索算法的规则，即根据输入的 GA/PSO 算法选择概率，随机选择下一步进行遗传算法或者粒子群算法。在两个子搜索算法中，GA 中的个体于 PSO 中的粒子是同一概念，表示各个决策解向量；

GA 中的种群于 PSO 中的粒子群是同一概念，表示解的集合。

步骤 5：执行子搜索算法：执行子搜索算法是多目标优化问题求解过程中的一个关键步骤，它涉及到对当前解集进行评估、改进和更新，以寻找更优的解。如果子搜索算法是遗传算法，则将对微服务策略解执行快速非支配排序、选择操作、交叉与变异操作、生成下一代种群。如果子搜索算法是粒子群算法，则将执行更新策略解粒子的速度和位置、选择全局最优策略解、更新最优解外部仓库、生成下一代粒子群。

步骤 6：循环迭代：重复步骤 2 至步骤 5，直至满足算法的终止条件。循环结束算法将后返回存有帕累托前沿解集的最优解外部仓库。本文算法的终止条件为到达设定的迭代次数。

其算法流程图如下：

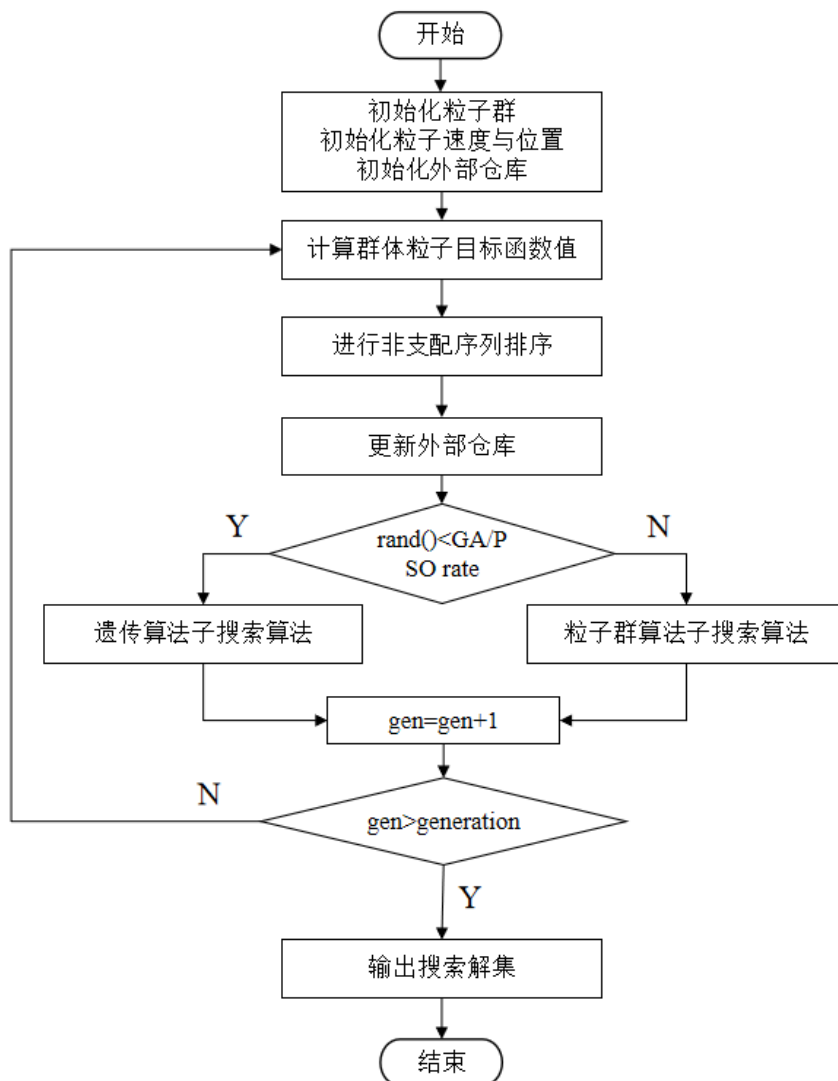


图 4.1 MOGA-PSO 算法流程图

算法伪代码如下：

算法 4.1 MOGA-PSO 双启发式算法

输入：种群数量，`pop_size`；迭代次数，`generation`；GA 算法参数，交叉率与变异率；PSO 算法参数，`w`、`c1`、`c2`；GA/PSO 选择比率，`GA/PSO rate`；

输出：近似帕累托前沿部署策略解，最优解外部仓库 `archive`

```

1: 初始化种群 population，初始化粒子速度与位置；
2: 初始化外部仓库 archive；
3: for gen←0 to generation do
4:     计算种群中每个个体的目标函数值 fitnesses；
5:     根据目标函数值获得非支配排序 sorted_fronts；
6:     将处于非支配序列第一层的粒子放入外部仓库 archive；
7:     保留外部仓库 archive 的非支配解；
8:     if rand()<GA/PSO rate then
9:         执行遗传子搜索算法，更新种群；
10:    else
11:        执行粒子群子搜索算法，更新种群；
12:    end if
13:    gen←gen+1；
14: end for
15: 保留外部仓库 archive 的非支配解；
16: return archive

```

4.2 详细设计

4.2.1 编码规则

本文在对自变量编码时采用整数编码的方式。假设一个机器集群中有 d 个节点，微服务集合中有 m 个微服务，则部署决策自变量 $\mathbf{x}$ 为大小为 m 的数组，每个位置取值 $[1, d]$ 之间的整数，数组每一位表示这一位代表的微服务所部署的节点标号。如图 4.2，对于 $\mathbf{x} = [4, 3, 2, 1, 4, 4]$ 的决策向量，表示微服务 `ms1` 部署在节点 `node4`，

微服务 ms2 部署在节点 node3, 微服务 ms3 部署在节点 node2, 微服务 ms4 部署在节点 node1, 微服务 ms5 部署在节点 node4, 微服务 ms6 部署在节点 node4。

用这种方式编码的好处在于自动实现了每个微服务实例被部署在一个节点上的约束, 能够有效降低问题求解的复杂程度。且这种编码方式能够很好的应用于遗传算法的交叉变异操作和粒子群算法的位置更新操作, 进一步降低编程复杂程度, 提高算法效率。

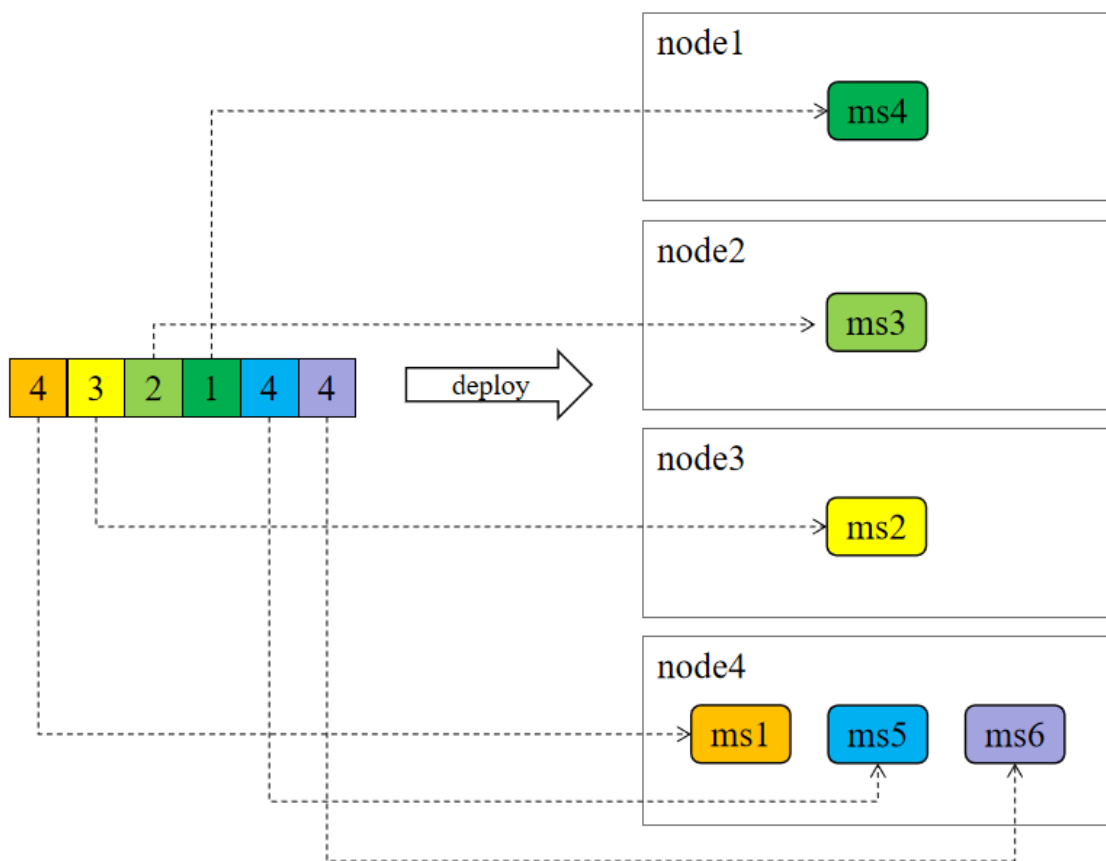


图 4.2 决策向量的部署

4.2.2 适应度函数

对于本算法中的适应度函数, 参考 3.2 节目标函数中的微服务网络延时开销与集群负载均衡性能。适应度函数是度量个体优劣的程度的函数, 由于微服务网络延时开销与集群负载均衡性能的值均为越小越优, 以此在求解微服务部署策略问题中, 算法认为适应度函数的值越小, 个体适应度越好。由于本文的微服务部署问题是一个多目标优化问题, 因此在本算法中适应度函数返回值也是一个多元数组, 第一个值为微服务网络延时开销, 第二个值为集群负载均衡性能。

模型中对问题的约束条件在适应度函数中采用惩罚项来表示。在计算个体适

应度值之前, 会先判断该自变量向量 $\mathbf{x}$ 是否满足了约束, 如果未满足约束条件, 则对所有适应度的所有维度加上一个惩罚项, 惩罚项值远大于正常值, 因此在实际搜索过程中, 该解将会以极高的概率淘汰掉。

4.2.3 遗传算法部分

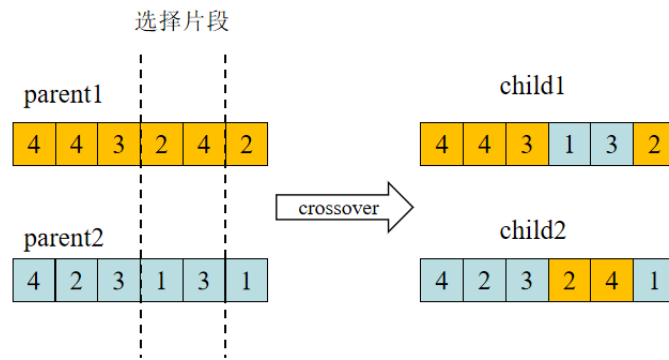
遗传算法部分基于 NSGA 算法, 输入为上一代种群、种群数量、非支配序列排序、交叉率、变异率。进入遗传算法子搜索算法后, 将会根据非支配序列排序对种群进行选择操作, 然后根据交叉率、变异率参数执行交叉、变异操作。经过这三种搜索操作后, 生成的新种群与前种群的种群数目应保持相同, 以防止出现种群越来越多或者越来越少的现象, 影响算法性能。

选择操作采用精英保留策略, 选择当前群体中属于非支配序列第一层的个体, 复制一份样本, 使其不参与后续的交叉变异操作直接传递到下一代。其余个体在进行交叉变异操作后再加入到新一代群体中。这种策略确保每一代中至少有一部分优秀个体能够被保留下来, 防止出现退化现象, 即由于交叉变异的随机操作丢失了一些重要的遗传信息, 使得群体的多样性降低, 影响算法的搜索能力和收敛速度。

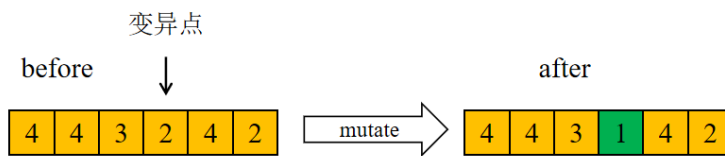
交叉操作采用两点交叉, 方式为随机从群体中选择两个父代个体, 随机选择个体向量中的两个交叉点, 然后交换两个父代个体在这两个点之间的部分。交叉操作的含义是随机把一个部署策略的相邻几个微服务部署节点与另一个部署策略相同微服务。如图 4.3(a)所示, 将 *parent1* 与 *parent2* 个体的 4-5 号位片段交换, 得到了 *child1* 与 *child2* 两个新个体。良好的交叉操作可以让陷入局部最优解的部署策略跳出局部最优, 寻找新的更优的策略。

变异操作采用单点随机均匀重置变异, 简称单点变异, 方式为随机选择个体的一个基因位, 并给它们重新替换一个限制范围内的均匀随机整数值。变异操作的含义是把一个部署策略某一个微服务所部署的节点随机替换为另一个节点。如图 4.3(b)所示, 将 *before* 个体的 4 号微服务部署的 2 节点变异为 1 节点, 生成新个体 *after*。变异操作通过一个部署策略微服务部署节点进一步扩大搜索范围, 使得搜索进入到一些新的解空间种。交叉与变异操作是 GA 算法能够在全局范围搜索的关键, 通过这两种随机操作, 可以使得算法能够探索更多解空间, 防止 GA 算法陷入局部最优解。

对于实际问题需要进一步的参数调节, 以获得更佳的算法性能。



(a) 交叉操作



(b) 变异操作

图 4.3 遗传算法操作

算法伪代码如下：

算法 4.2 GA 子搜索算法

输入：种群数量，pop_size；种群，population；GA 算法参数，交叉率与变异率；

非支配序列排序，sorted_fronts；

输出：下一代种群，population

1: 根据 sorted_fronts 进行选择操作，结果暂存入 selected_population；

2: 初始化下一代种群集合 next_population；

3: **while** (len(next_generation)<pop_num) **do**

4: 随机从 selected_population 选取两个体作为下一代父类，暂存到 parent1，parent2；

5: 用 parent1，parent2 进行交叉操作，结果暂存到 child1，child2；

6: 对 child1，child2 进行变异操作；

7: 将 child1，child2 存入 next_population；

8: **end while**

9: population←next_population；

10: **return** population

另外,对于本文的 MOGA-PSO 算法中的遗传算法部分,考虑到微服务部署问题的特性,本文选择采用 NSGA 算法,而非 NSGA-II 算法。NSGA-II 与 NSGA 算法的区别在于前者在选择时会考虑解集中各个解之间的拥挤距离,但是通过实际测试发现,在微服务部署问题中帕累托前沿解之间拥挤程度不高,无需使用拥挤距离来进一步选择解集中的解,因此为了简化算法流程,同时提高算法迭代速度,本文采用 NSGA 算法作为遗传算法部分。

4.2.4 粒子群算法部分

粒子群算法部分基于 MOPSO 算法,输入为上一代粒子群、粒子群数量、非支配序列排序、惯性权重 w 、个体最优速度权重 c_1 、全局最优速度权重 c_2 、最优解外部仓库、粒子速度向量。进入粒子群算法子搜索算法后,将会根据 w 、 c_1 、 c_2 参数执行粒子速度、位置更新操作,然后更新粒子历史最佳位置。生成的新粒子群与前粒子群的粒子群数目应保持相同。

速度更新操作时,会对每一个粒子根据最近粒子优先原则,从外部仓库中选择离该粒子欧拉距离最短的最优粒子作为该粒子的全局最优解,用于更新粒子的速度和位置。这个选择过程有助于粒子聚焦于搜索空间中那些已经被证明是较优解的区域。速度更新方式见公式 2.5,其本质是利用全局最优解、本地最优解、速度惯性信息计算出微服务部署策略中每个微服务部署节点的更改趋势,这种趋势指引粒子快速收敛到最优解。

位置更新操作是直接将当前位置加上当前速度,然后四舍五入取整进入下一代,详情见公式 2.6。这个取整过程是必要的,因为在微服务部署策略中,部署节点的位置通常需要是整数,代表具体的服务器或容器实例。位置更新操作的作用是使得部署策略粒子根据更改趋势进行微服务部署节点的转移,实现更快速的寻找到优秀部署策略。粒子群算法的速度位置更新操作使得该算法迅速收敛到函数的极值。

位置更新操作可能会使得粒子超出有效范围内,因此需要使超出范围的粒子回到正常范围内以便进一步运算。本算法中会将处于上下限外的粒子的值赋值为上下限。

最后需要更新每个粒子的本地最优解,如果更新后的粒子帕累托支配原先本地最优解,则更新本地最优解。

同样的，对于实际问题需进一步参数调节，以获得更佳的算法性能。

算法伪代码如下：

算法 4.3 PSO 子搜索算法

输入:粒子群数量, pop_size ; 粒子群, population ; PSO 算法参数, w 、 c_1 、 c_2 ;

外部仓库, archive ; 粒子的速度向量;

输出: 下一代种群, population ;

1: 初始化下一代种群集合 next_population ;

2: **for** $i \leftarrow 0$ **to** pop_size **do**

3: 根据 archive 为粒子 $\text{population}[i]$ 选择领导解 leader ;

4: 更新粒子的速度向量;

5: 更新粒子的位置向量;

6: 判断粒子是否再有效范围内, 对于超出边界的值使其等于边界值;

7: 更新粒子的本地最优解;

8: **end for**

9: $\text{population} \leftarrow \text{next_population}$;

10: **return** population

第五章 实验部分

5.1 实验设置

5.1.1 实验环境

为了评估与验证算法有效性，本文在物理机上搭建了一套由 VMWare 配置的多个虚拟机组成的 Kubernetes 集群。物理机与虚拟机节点详细信息见表 5.1、表 5.2。

表 5.1 物理机硬件配置

硬件名称	设备型号
CPU	Intel Core i5 10300H
内存	32G
硬盘	1T

表 5.2 虚拟机配置

虚拟机节点	CPU	内存	操作系统
master	2core	2Gb	CentOS7.9.2009
node1	2core	4Gb	CentOS7.9.2009
node2	2core	4Gb	CentOS7.9.2009
node3	2core	2Gb	CentOS7.9.2009
node4	4core	4Gb	CentOS7.9.2009

Kubernetes 虚拟机集群中的每台虚拟机的软件配置如表 5.3 所示。

表 5.3 虚拟机软件配置

软件名称	版本号
Kubernetes	1.28.2
Docker	24.0.7
Cri-docker	0.3.9-3
Flannel	0.24.0
Istio	1.18.2

子节点虚拟机之间的网络延时见表 5.4，这是通过 VMWare 软件中对虚拟机的网络设配器中虚拟机传入与传出传输的延时数进行设置。

表 5.4 虚拟机之间的网络延时（单位：ms）

	node1	node2	node3	node4
node1	0	43	32	50
node2	43	0	24	42
node3	32	24	0	34
node4	50	42	34	0

5.1.2 测试微服务程序

本文选用 Bookinfo 微服务来进行实机部署，详细信息见 2.3.2 Bookinfo 章节。

Bookinfo 应用程序的微服务之间的调用关系、微服务资源消耗量如表 5.5 所示。调用关系通过 Bookinfo 微服务调用图与 Jaeger 分布式跟踪系统获得，每个微服务的资源需求量通过在 Kubernetes 集群中执行 kubectl top pod 命令来获取每个微服务的 CPU 消耗量与内存消耗量。通过 Jaeger 分布式跟踪系统去监控 Bookinfo 微服务，其服务调起过程为：请求发送到入口微服务 productpage 后，productpage 会先调用 details 获取图书详情，将数据返回给 productpage 后 productpage 会调用 review 的 3 个版本之一来获取图书评价，若调用了 review 的 v1 版本，则返回数据后服务结束；若调用了 review 的 v2 或 v3 版本，review 还会继续调用 ratings 获得相应图书星级并按链条返回。因此该微服务包含 3 条调用链，如图 5.1 所示，图中 details 到 productpage’为表示 details 并非调起了 productpage，而是给 productpage 返回数据继续调用之后的微服务。本文设置三条调用链的请求概率相等，均为 $\frac{1}{3}$ 。

表 5.5 微服务详细信息

名称	id	调用关系	CPU 消耗量 (mCore)	内存消耗量 (Mb)
productpage	ms1	{ms2,ms3,ms4,ms5}	1.2	200
reviews-v1	ms2		0.05	300
reviews-v2	ms3	{ms6}	0.2	200
reviews-v3	ms4	{ms6}	0.5	300
details	ms5		0.15	60
ratings	ms6		0.05	100

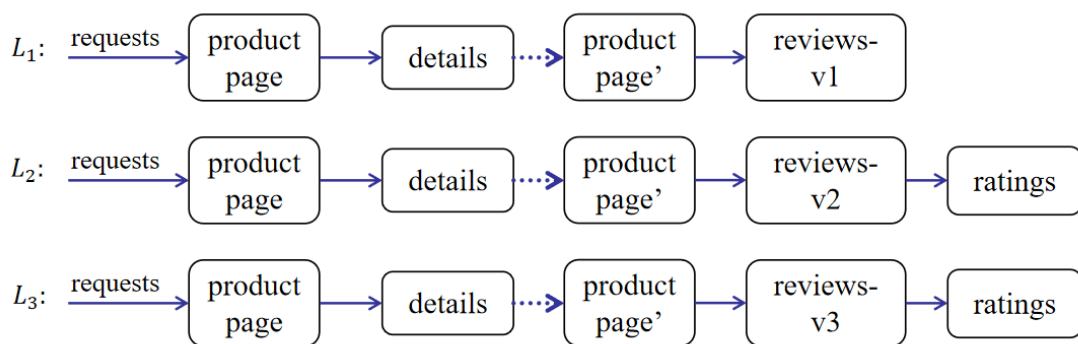


图 5.1 Booinfo 微服务调用链集合

5.1.3 算法对比

为了验证本文所提出的 MOGA-PSO 的在搜索结果与部署策略方面的优势，本文将 NSGA、MOPSO 作为对比算法，来比较三个启发式算法在搜索性能上的优势。三种算法均由 Python 语言编写，程序运行过程会记录算法迭代过程中产生的详细数据，包括迭代次数、每次迭代解集等，用于进行算法过程分析。

在对比三种算法在实机测试中的效果时，由于这三种算法都是多目标优化算法，算法运行结果是一个含有多个互相不支配的帕累托前沿解集合，因此需要按照某种策略从中选出一个代表解来代表算法最优解去部署在实机集群中进行测试。本文选用的策略是最高加权平均分选用策略，在该策略下会为解的每个目标值分配权重系数 ω ，然后按照公式 5.1 为解集中全部解进行打分，选出分数最高的解作为该算法的代表解。该公式中 S 为解集的目标值集合， m 代表目标数量。

$$score = \left(\sum_{i=1}^m \omega_i \frac{(S_i - S_i^{min})}{S_i^{max} - S_i^{min}} \right)^{-1} \quad (5.1)$$

在实机测试环节中，实机测试中本文还引入了 Kubernetes 原生策略作为基准性能参考。对于微服务请求响应时间的测量，本文使用了 JMeter，一个广泛用于负载测试和性能测量的轻量级测试平台，对不同策略部署的 BookInfo 微服务进行负载测试。JMeter 可以通过定制线程的数量来模拟大量的用户并发请求，并监控请求访问的细节。本文设置每组 1、10、50、100、200、500、1000 每次的并发请求实验，在 Jmeter 软件界面收集网络延时的平均值和 P95 尾部延时。对于集群资源均衡率的测量，本文用 Kubernetes 的基础命令 `kubectl top node` 来获取，该命令会返回集群中各个节点的 CPU、内存使用量与使用率。执行该命令需要提前部署 `metric-server` 组件。本文设置在稳定的 100 每次的并发请求下，使用该命令测试集群节点

的资源使用量。

5.1.4 实验参数

启发式算法参数见表 5.6 所示。

表 5.6 启发式算法参数

算法名称	种 群 数量	迭代次数	交叉率	变异率	C1	C2	w	GA/PSO rate
NSGA	50	3000	0.2	0.08	/	/	/	/
MOPSO	50	3000	/	/	1.5	1.5	0.5	/
MOGA-PSO	50	3000	0.2	0.08	1.5	1.5	0.5	0.99

表 5.7 最高加权平均分选用策略权重

参数名称	数值
ω_1	0.5
ω_2	0.5

5.2 实验结果与分析

本节将展示 NSGA、MOPSO 和 MO-GA-PSO 生成的帕累托前沿，并对比三种算法的收敛性能。同时通过实机测试微服务网络延时、集群 CPU 占用率标准差、集群内存占用率标准差，来比较 Kubernetes 原生策略、NSGA、MOPSO 和 MO-GA-PSO 之间的差异，并分析差异的原因。

5.2.1 帕累托前沿

对于本文的三种启发式多目标优化问题，所得到的解集是真实帕累托前沿的子集或者帕累托前沿近似解，解越靠近左下角越优。三种启发式多目标优化问题得到的帕累托前沿结果见图 5.2。

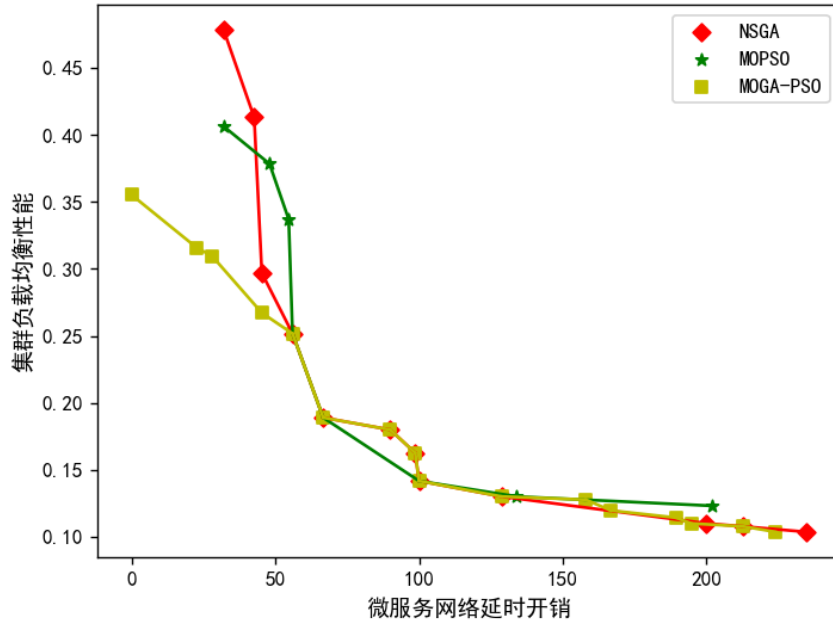


图 5.2 帕累托前沿解

从图 5.2 中可以看出，在微服务网络延时开销目标 0 到 50ms 部分，MOGA-PSO 搜索到的解靠近左下角，支配 NSGA 与 MOPSO 的解；在微服务网络延时开销目标 50 到 200ms 部分，三种算法都可以搜索到近似帕累托前沿解；在微服务网络延时开销目标 200ms 以上部分，MOGA-PSO 算法与 NSGA 算法可以搜索到部分解。从整体来看，MOGA-PSO 搜索到的解集更接近与真实帕累托前沿，MOPSO 与 NSGA 算法在个别部分也可以搜索到良好的帕累托前沿解。

5.2.2 收敛性能

本文用搜索过程中每次迭代产生的结果的迭代距离 (Generational Distance, GD) 与反向迭代距离 (Inverted Generational Distance, IGD) 来评估算法的收敛性能^[39]。

迭代距离可以衡量真实帕累托前沿解集与多目标优化算法所搜索出的近似解集之间的间隔距离。设 PF 为真实帕累托前沿， S 为近似解集，则迭代距离 GD 定义如下：

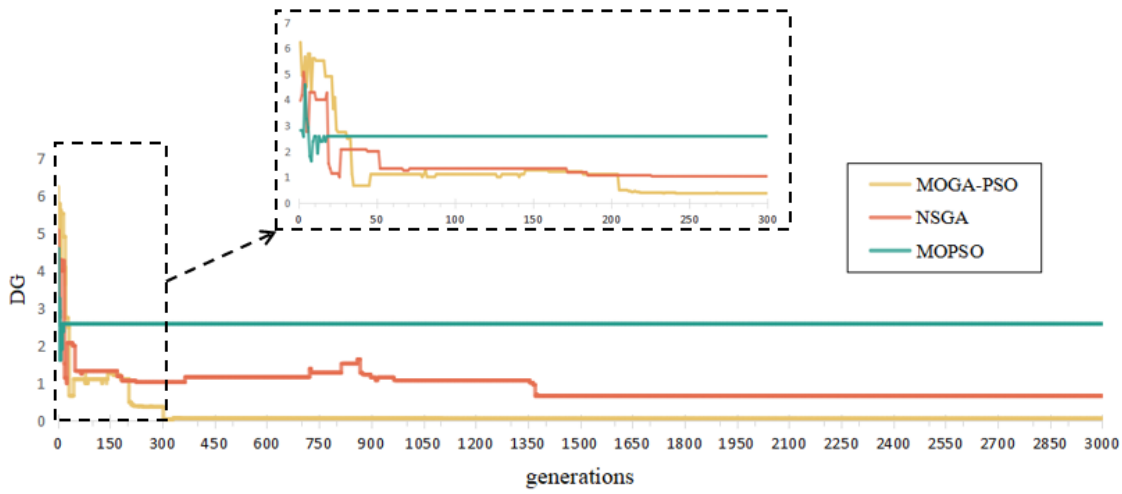
$$GD(S, PF) = \frac{\sqrt{\sum_{x \in S} dist(x, PF)^2}}{|S|} \quad (5.2)$$

其中， $dist(x, PF)$ 表示个体 x 到 PF 上离其最近个体之间的欧氏距离， $|S|$ 是集合 S 的基数。 GD 值越小，表示 S 具有越好的收敛性，越能逼近整个 PF 。当 S 为 PF 的子集时， GD 值将会等于 0，此时无法评估该解集是否还能进一步收敛，因此本文又引出反向迭代距离来评估算法收敛性。

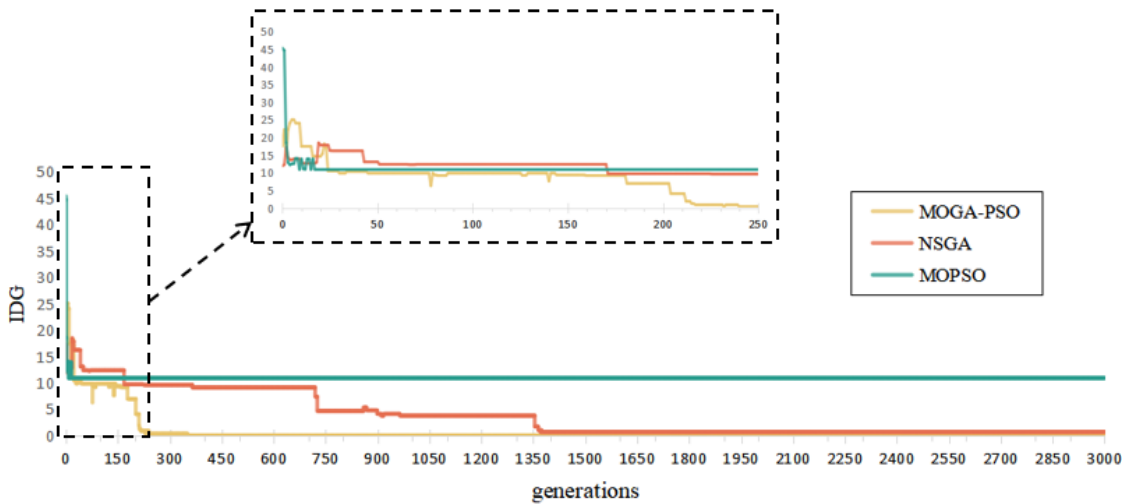
反向迭代距离可以衡量的是真实 PF 中的个体到算法搜索的近似解集之间的间隔距离。反向迭代距离 IGD 定义如下：

$$IGD(S, PF) = \frac{\sqrt{\sum_{x \in PF} dist(x, S)^2}}{|PF|} \quad (5.3)$$

IGD 同样可以评估解集收敛性。 IGD 不同于 GD 的是，当 S 为 PF 的子集，即 $S \subset PF$ 时， $IGD \neq 0$ ，且随着 S 中属于 PF 的解增多， IGD 指标越趋近于 0，直到 $S = PF$ 时， $IGD = 0$ 。因此 IGD 可以进一步评估 S 与 PF 之间的距离，以此观察算法收敛速度。



(a) DG 迭代图



(b) IDG 迭代图

图 5.3 三种算法的迭代性能

三种搜索算法 GD 与 IGD 迭代图如图 5.3(a)、图 5.3(b)所示。由这两张图可知，MOPSO 算法收敛速度快，可以在 50 步以内收敛，但是收敛后会陷入局部最优，

不再进一步向帕累托前沿靠近;NSGA 算法收敛时间长,在 1300 余步才开始收敛,但是可以更近一步向着帕累托前沿靠近,所得到的解效果较好;MOGA-PSO 算法收敛速度较快,在 300 余步即可收敛,且可以解集与帕累托前沿距离更近,收敛性能最佳。该结果说明 MOGA-PSO 算法在收敛速度相对于 NSGA 算法优化了 78.5%。在 GD 指标上 MOGA-PSO 相对于 NSGA 与 MOPSO 分别优化了 90.0%与 97.4%,在 IGD 指标上分别优化了 94.1%与 99.6%,表明 MOGA-PSO 在解集质量上相对于 NSGA 于 MOPSO 也有大幅度提升。

5.2.3 微服务请求响应时间

微服务请求响应时间指的是向服务器发送请求到获得全部响应的时间间隔,是判断部署策略性能的重要指标之一。请求响应时间会受到多个因素的影响,如请求并发数、传输数据量大小、节点拓扑关系等。

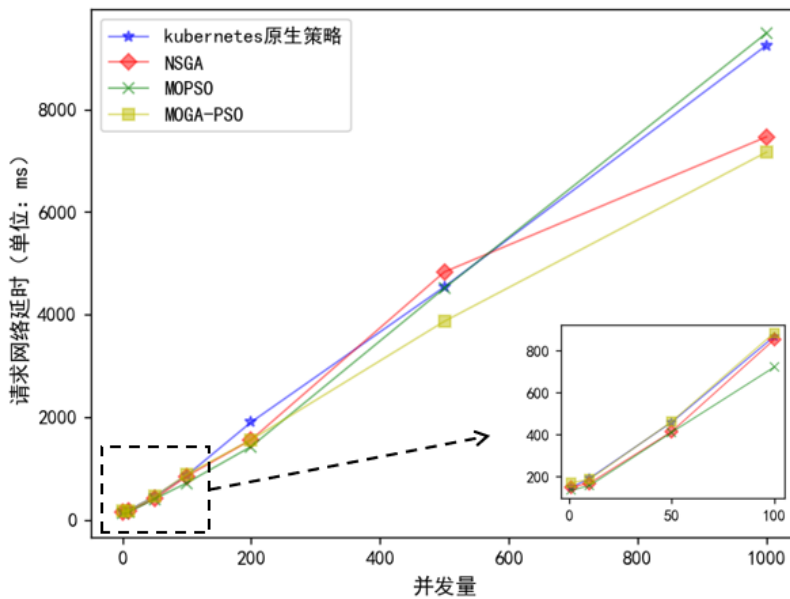
本文通过测试在相同微服务集合、相同节点资源与拓扑关系,不同请求并发数的下,不同微服务部署策略对于请求响应时间的影响,以此来对比评估部署策略的性能。MOGA-PSO、NSGA、MOPSO 算法产生的部署策略采用式 5.1 对多目标优化算法搜索结果采用最高加权平均分选用策略选出,参数见表 5.7。

请求响应时间测试结果由 Jmeter 软件给出,其中,平均延时指的是所有请求的延时时间的平均值,反映系统的总体性能;P95 尾部延时指的是在所有请求中,最慢的 5%的请求的平均延时,该指标反映系统的性能瓶颈。对于测试结果分析上,平均延时和 P95 尾部延时越小,说明算法的调度方案越优,请求响应性能越好。图 5.4 分别展示了系统响应时间的平均值和 P95 尾部延时。

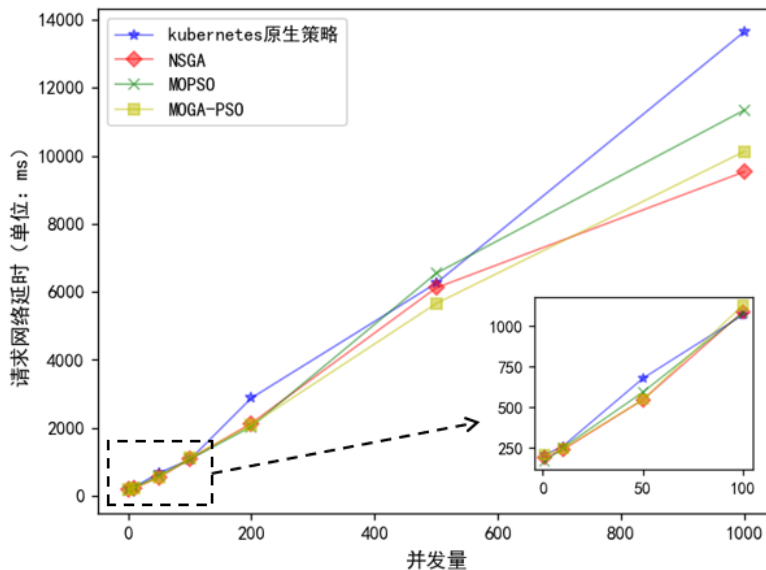
从图 5.4(a)与图 5.4(b)可以看出在小于 100 每次的低并发请求下,3 种启发式算法与默认算法的响应时间相近,MOGA-PSO 算法的平均响应时间略慢一些,其平均延时相较于 Kubernetes 原生策略减慢了 4.0%,P95 尾部延时加快了 5.0%,这仅仅是轻微幅度的波动;在高于 100 每次的高并发请求下,MOGA-PSO 算法与 NSGA 算法的响应时间更低,优于 MOPSO 算法与 Kubernetes 原生策略。MOGA-PSO 较于 Kubernetes 原生策略,其平均延时加快了 18.4%,P95 尾部延时加快了 20.9%。这表明了本文的 MOGA-PSO 算法在高并发请求中能提供给用户更快的相应,验证了算法有效性。

其原因在于 Kubernetes 原生策略仅考虑微服务消耗资源、节点资源等信息,

在本文对于微服务部署问题的模型在考虑资源消耗的基础上，还考虑了微服务相互调用产生的网络延时开销影响和集群负载均衡影响，优化了链路之间的调起流程，并优化了资源配置，因此能够有效降低响应请求时间，提高了 QoS。由于 MOPSO 算法未收敛到一个合适的程度，因此其部署效果不如 MOGA-PSO 与 NSGA 算法，但其仍在一定程度上有优化效果，适用于对部署反应速度有极高要求的场景。MOGA-PSO 与 NSGA 算法由于其最终收敛程度相近，因此两者的部署效果相近。MOGA-PSO 在收敛速度上远高于 NSGA 算法，因此在实际部署时也会有更快的部署速度。



(a) 响应时间平均值指标



(b) 响应时间 P95 尾部延时指标

图 5.4 不同并发量请求下的请求响应时间

5.2.4 集群资源均衡率

集群资源均衡率也是衡量部署策略的一个重要指标，他体现了集群中资源分布是否被均衡的使用。标准差是概率统计中常用的离散度测量方法，用于反映群体内个体之间的离散程度。本文通过集群的 CPU 资源和内存资源占用率的标准差作为集群资源均衡率的性能指标。标准差值越大，表示节点资源使用越不稳定；标准差值越小，表示集群节点资源配置越均匀。图 5.5 展示了四种策略在集群资源均衡率上的性能对比。

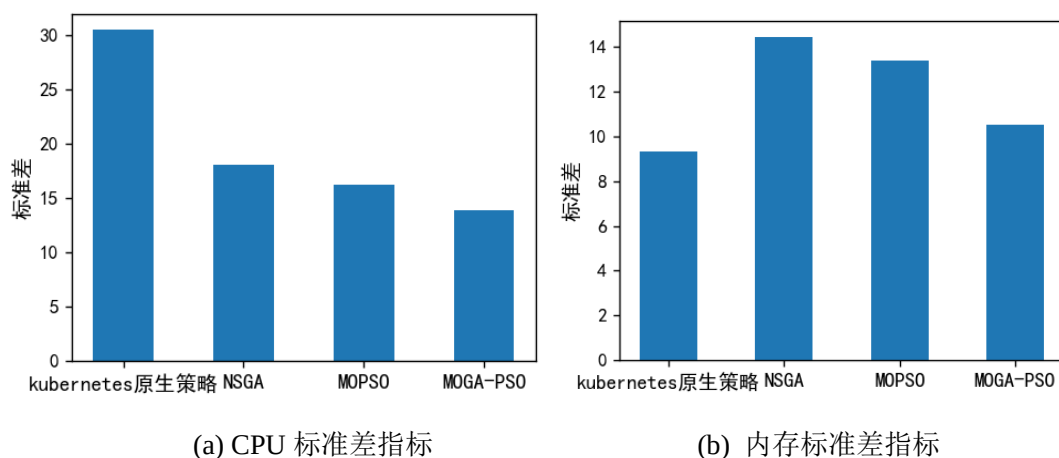


图 5.5 集群资源均衡率

如图 5.5(a)，对于 CPU 资源均衡率指标，NSGA、MOPSO、MOGA-PSO 算法分别相较于 Kubernetes 原生策略优化了 40.6%、46.8%、54.4%。三种启发式算法均起到了一定的优化作用，能够有效降低对 CPU 的资源利用率的标准差，提高 CPU 均衡率。在三种启发式算法中，MOGA-PSO 算法对 CPU 占用率标准差的优化效果最好。

如图 5.5(b)，对于内存资源均衡率，三种启发式算法部署结果均劣于 Kubernetes 原生策略，标准差指标分别升高了 54.7%、43.1%、12.7%。在三种启发式算法中，NSGA 算法对内存占用率标准差的影响最大，MOGA-PSO 对内存占用率标准差影响较其他两种算法较小。

分析其原因，Kubernetes 原生策略与本文模型均考虑了节点资源与微服务消耗资源。Kubernetes 原生策略部署时会每个 pod 均匀地分配给每个物理节点，因此在资源均衡率上可以得到一个良好的结果。本文模型同时考虑了微服务调用产生的网络延时开销与微服务部署时的节点资源均衡率，这两个目标相互制约。因此对于集群资源均衡率，不同策略会在不同类型的资源上产生较好的结果。启发式算法

得出的部署结果与其收敛程度、对于帕累托前沿解的选择有关，其在总体上越靠近帕累托前沿，能获得的性能越优秀。从结果上看，基于微服务部署模型的启发式算法牺牲了部分内存资源均衡率，获得了更快的请求响应时间。

第六章 总结与展望

由于传统单体架构应用可扩展性弱，故障影响范围大，不适用于功能复杂的大型应用，因此微服务架构被提出。微服务架构将传统单体业务拆分，形成多个可相互调用的模块化微服务。将这些微服务分布式的部署到不同的机器节点上为用户提供服务，具有灵活、可扩展、高可用的优势。但在其实际应用中，仍然存在许多需要解决的问题，其中一个关键问题就是微服务实例部署优化问题。该问题需要考虑到服务器集群硬件资源、网络延迟和应用可靠性等多个方面，涉及的因素很多，有大量的决策变量和约束条件，因此对其直接求解的计算复杂度很高。

本文通过将遗传算法与粒子群融合，提出了 MOGA-PSO 双启发式算法，针对微服务部署策略问题进行了研究。首先通过分析微服务的原理特性，构建基于多目标优化的微服务部署问题模型，明确问题的目标函数和约束条件；然后对 NSGA 与 MOPSO 进行了融合与改进，使得该算法能够很好的应用于微服务部署问题求解；最后通过实验对算法结果、搜索性能、实机部署测试多个方面进行综合评估，并对比传统启发式算法与 Kubernetes 原生策略，该算法能够更快速的搜索到更优秀的解，提升了搜索效率；其在实机部署时也能够有效优化在不同并发量下的服务请求相应时间，证明本文的 MOGA-PSO 算法对微服务部署问题的求解有一定程度的优化作用，能够有效优化高并发状态下的微服务请求响应时间。

本文存在的问题在于对在获取数据时，采用的 Kubernetes 自带的资源获取方式可能无法获取到节点最新的资源状况，且模型中微服务消耗资源量也是根据先验知识和先前的测试结果来计算的，因此模型结果可能与实际部署情况有差距。模型中对于网络也并未考虑带宽对请求的限制影响，实际上网络带宽也会影响在高并发请求下服务器对外服务。

在未来可以考虑对于模型进行改进，例如考虑网络带宽等更多变量对微服务 QoS 的影响，优化对微服务部署问题的求解。也可以从获取数据方面考虑，利用更准确的测量方式获取集群的如资源消耗量等参数，进一步提高算法准确性，提升部署性能。

致谢

随着这篇本科毕业设计的完成，我四年的大学生活也即将画上句号。在这个重要的时刻，我想向那些在我求学路上给予我无私帮助和支持的人表达我最深的谢意。

首先，我要感谢我的导师。您严谨的学术态度、深厚的专业知识，让我在学术道路上受益匪浅。从选题到实验设计，从数据分析到论文撰写，每一步您都给予了我耐心的指导和细致的修改。您的严格要求让我更加明白学术的严谨性和重要性，也让我在面对困难时更加坚韧不拔。您的言传身教，将是我未来学术道路上最宝贵的财富。

其次，我要感谢我的学长。在毕业设计的过程中，你们也给予了我很多宝贵的建议和经验，让我少走了很多弯路。您的支持和鼓励，让我在遇到困难时更加有信心和勇气去面对。

我还要感谢我的父母。是你们一直以来的支持和理解，让我能够全身心地投入到学习和研究中。你们为我提供了温暖的家庭和舒适的学习环境，让我能够无忧无虑地追求自己的梦想。你们的付出和辛劳，我将永远铭记在心。

最后，我要感谢我的朋友们。是你们陪我度过了大学四年的美好时光，一起经历了欢笑和泪水。在毕业设计的过程中，你们给予了我很多帮助和鼓励，让我感受到了友谊的温暖和力量。我会珍惜这份友谊，让它成为我人生中最宝贵的财富。

在此，我再次向所有给予我帮助和支持的人表示衷心的感谢。我将带着你们的期望和祝福，继续前行，在学术和人生的道路上不断追求进步和成长。

参考文献

- [1] Abdullah M, Iqbal W, Erradi A. Unsupervised learning approach for web application auto-decomposition into microservices[J]. Journal of Systems and Software, 2019, 151: 243-257.
- [2] Villamizar M, Garces O, Ochoa L, et al. Infrastructure cost comparison of running web applications in the cloud using AWS lambda and monolithic and microservice architectures[C]//2016 16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid). IEEE, 2016: 179-182.
- [3] Piccialli F, Benedusi P, Amato F. S-InTime: A social cloud analytical service oriented system[J]. Future Generation Computer Systems, 2018, 80: 229-241.
- [4] Daya S, Van Duy N, Eati K, et al. Microservices from theory to practice: creating applications in IBM Bluemix using the microservices approach[M]. IBM Redbooks, 2016.
- [5] Ren Z, Wang W, Wu G, et al. Migrating web applications from monolithic structure to microservices architecture[C]//Proceedings of the 10th Asia-Pacific Symposium on Internetware. 2018: 1-10.
- [6] Di Francesco P, Lago P, Malavolta I. Migrating towards microservice architectures: an industrial survey[C]//2018 IEEE international conference on software architecture (ICSA). IEEE, 2018: 29-2909.
- [7] 杨萍. 基于进化多任务优化的多目标容器部署算法研究[D]. 西安电子科技大学, 2022. DOI:10.27389/d.cnki.gxadu.2022.002233.
- [8] Xu M, Song C, Ilager S, et al. Coscal: Multifaceted scaling of microservices with reinforcement learning[J]. IEEE Transactions on Network and Service Management, 2022, 19(4): 3995-4009.
- [9] Prajapati A, Patel D M. Container scheduling: A taxonomy, open issues and future directions for scheduling of containerized microservices[J]. Open Issues and Future Directions for Scheduling of Containerized Microservices, 2024.
- [10] 林闯, 陈莹, 黄霁巍等. 服务计算中服务质量的多目标优化模型与求解研究[J]. 计算机学报, 2015, 38(10): 1907-1923.
- [11] Guan X, Wan X, Choi B Y, et al. Application oriented dynamic resource allocation for data centers using docker containers[J]. IEEE Communications Letters, 2016, 21(3): 504-507.
- [12] Zhou R, Li Z, Wu C. An efficient online placement scheme for cloud container clusters[J]. IEEE Journal on Selected Areas in Communications, 2019, 37(5): 1046-1058.

-
- [13] Rodrigues L R, Pasin M, Alves O C, et al. Network-aware container scheduling in multi-tenant data center[C]//2019 IEEE Global Communications Conference (GLOBECOM). IEEE, 2019: 1-6.
- [14] Lv W, Wang Q, Yang P, et al. Microservice deployment in edge computing based on deep Q learning[J]. IEEE Transactions on Parallel and Distributed Systems, 2022, 33(11): 2968-2978.
- [15] 张伟颖.基于元强化学习的容器化网络功能服务部署优化研究[D].华中科技大学,2022.DOI:10.27157/d.cnki.ghzku.2022.001893.
- [16] Computing M E. Delay-aware Microservice Coordination in Mobile Edge Computing: A Reinforcement Learning Approach[J]. IEEE TRANSACTIONS ON MOBILE COMPUTING, 2021, 20(3): 939.
- [17] Chen L, Xu Y, Lu Z, et al. IoT microservice deployment in edge-cloud hybrid environment using reinforcement learning[J]. IEEE Internet of Things Journal, 2020, 8(16): 12610-12622.
- [18] Xu M, Song C, Ilager S, et al. Coscal: Multifaceted scaling of microservices with reinforcement learning[J]. IEEE Transactions on Network and Service Management, 2022, 19(4): 3995-4009.
- [19] Tan B, Ma H, Mei Y. Novel genetic algorithm with dual chromosome representation for resource allocation in container-based clouds[C]//2019 IEEE 12th international conference on cloud computing (CLOUD). IEEE, 2019: 452-456.
- [20] Lin M, Xi J, Bai W, et al. Ant colony algorithm for multi-objective optimization of container-based microservice scheduling in cloud[J]. IEEE access, 2019, 7: 83088-83100.
- [21] 陆海锋,赵嘉凌,欧阳学名,等.云环境下实现容器部署的加速粒子群优化算法[J].计算机应用研究,2024,41(03):756-763.DOI:10.19734/j.issn.1001-3695.2023.07.0312.
- [22] ZHANG R, CHEN Y, DONG B, et al. A genetic algorithm-based energy-efficient container placement strategy in CaaS[J]. IEEE Access, 2019, 7 : 121360 – 121373.
- [23] 马武彬,王锐,王威超等.基于进化多目标优化的微服务组合部署与调度策略[J].系统工程与电子技术,2020,42(01):90-100.
- [24] SHI T, MA H, CHEN G. Multi-objective container consolidation in cloud data centers[C] // Australasian Joint Conference on Artificial Intelligence. 2018 : 783 – 795.
- [25] Zuo L, Shu L, Dong S, et al. A multi-objective optimization scheduling method based on the ant colony algorithm in cloud computing[J]. Ieee Access, 2015, 3: 2687-2699.
- [26] Chen X, Xiao S. Multi-objective and parallel particle swarm optimization algorithm for container-based microservice scheduling[J]. Sensors, 2021, 21(18): 6212.

-
- [27] Li H, Tang B, Xu W, et al. Application deployment in mobile edge computing environment based on microservice chain[C]//2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD). IEEE, 2022: 531-536.
- [28] KAEWKASI C, CHUENMUNEEWONG K. Improvement of container scheduling for Docker using ant colony optimization[C] // 2017 9th international conference on knowledge and smart technology (KST). 2017 : 254 – 259.
- [29] Li L, Chen J, Yan W. A particle swarm optimization-based container scheduling algorithm of docker platform[C]//Proceedings of the 4th International Conference on Communication and Information Processing. 2018: 12-17.
- [30] Seghir F, Khababa A. A hybrid approach using genetic and fruit fly optimization algorithms for QoS-aware cloud service composition[J]. Journal of Intelligent Manufacturing, 2018, 29: 1773-1792.
- [31] Hussein M K, Mousa M H, Alqarni M A. A placement architecture for a container as a service (CaaS) in a cloud environment[J]. Journal of Cloud Computing, 2019, 8: 1-15.
- [32] ADHIKARI M, SRIRAMA S N. Multi-objective accelerated particle swarm optimization with a container-based scheduling for Internet-of-Things in cloud environment[J]. Journal of Network & Computer Applications, 2019, 137 : 35 – 61.
- [33] Scrucca L. GA: A package for genetic algorithms in R[J]. Journal of Statistical Software, 2013, 53: 1-37.
- [34] Marini F, Walczak B. Particle swarm optimization (PSO). A tutorial[J]. Chemometrics and Intelligent Laboratory Systems, 2015, 149: 153-165.
- [35] 袁晓晨. 基于强化学习的微服务调度算法研究[D]. 中国石油大学(华东), 2024. DOI:10.27644/d.cnki.gsydu.2021.001368.
- [36] Kubernetes[EB/OL]. <https://kubernetes.io/zh-cn/>
- [37] Bookinfo[EB/OL]. <https://istio.io/latest/zh/docs/examples/bookinfo/>
- [38] Jaeger[EB/OL]. <https://www.jaegertracing.io/>
- [39] 胡涵,李振宇.多目标进化算法性能评价指标综述[J].软件导刊,2019,18(09):1-4.

